Berkeley MSSE
MOLECULAR SCIENCE & SOFTWARE ENGINEERING

Lecture 08:

Advanced Visualization Tools

Markus Hohle

University California, Berkeley

Data Science for Scientific Computing

MSSE 277A, 3 Units

Advanced Visualization Tools

credit: David C. Lovelace

Outline

UMAP	**just visualization**
t-SNE	
Leiden	**actual unsupervised learning, helpful for visualization**
DBSCAN	

Advanced Visualization Tools

credit: David C. Lovelace

Outline

UMAP

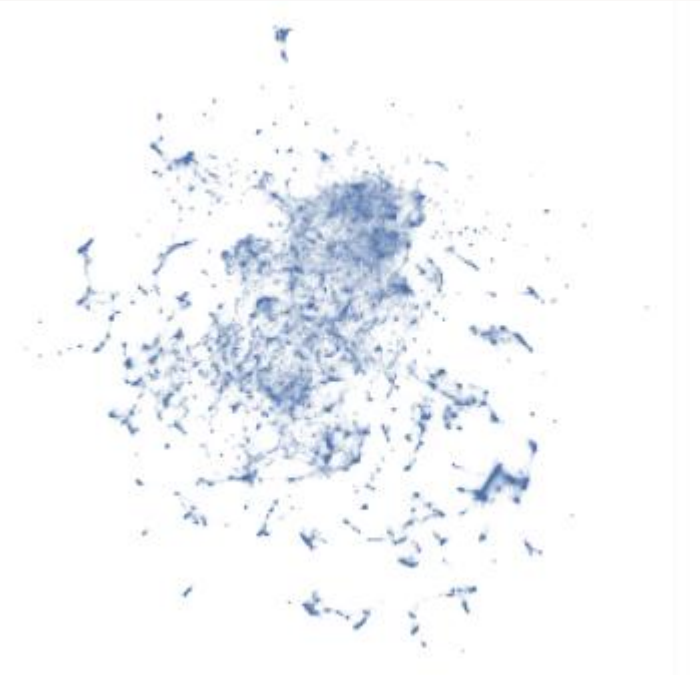
t-SNE

Leiden

DBSCAN

Advanced Visualization Tools:

UMAP

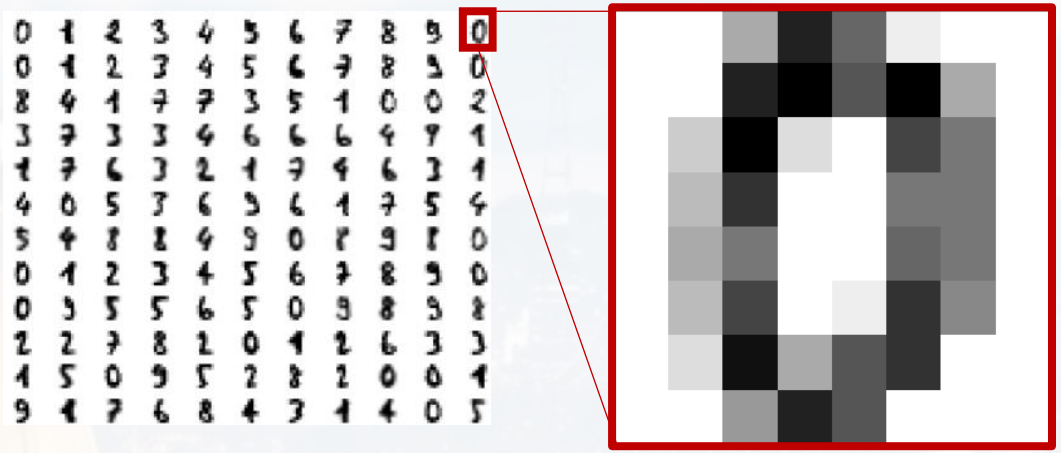


UMAP plot of word embedding

PCA:

+ simple, straight forward
- data lives on curved manifold

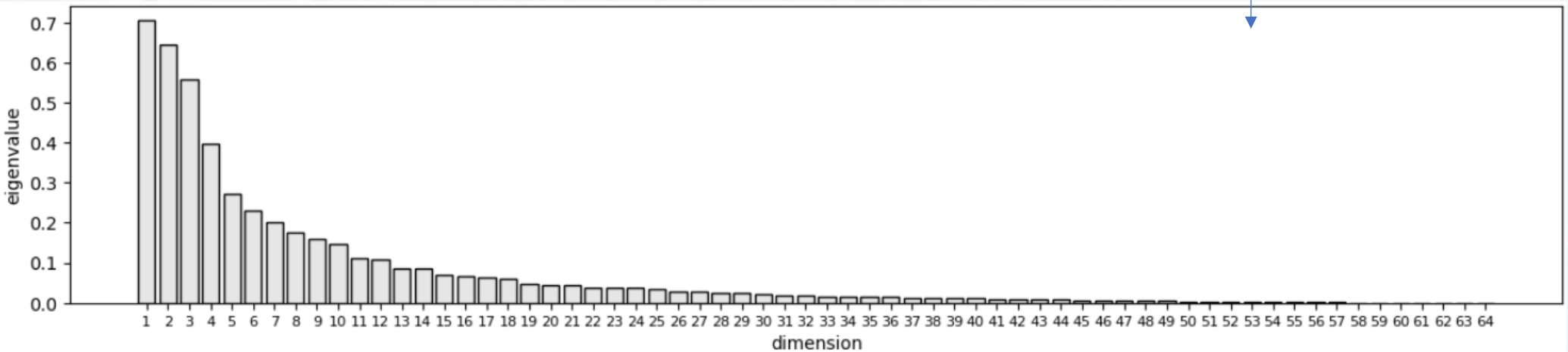
digits data set



8x8 gray scale images

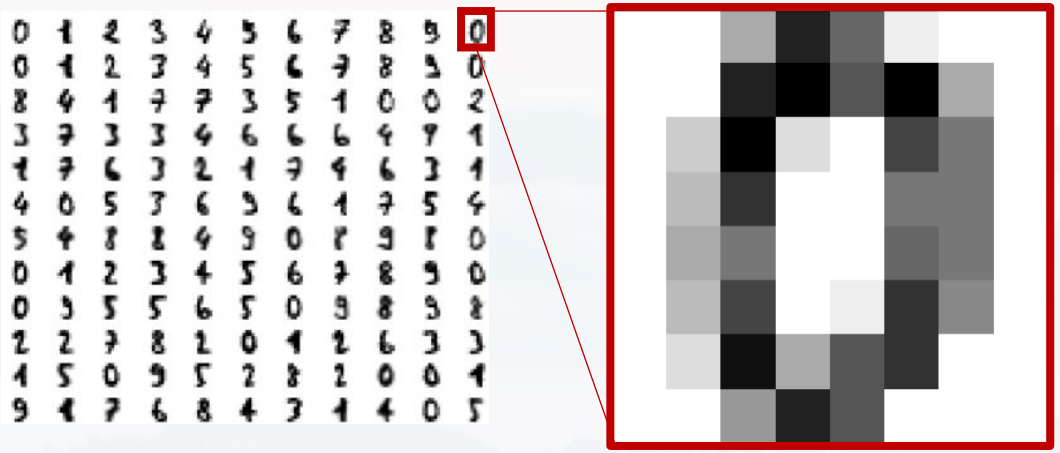
flattening:
64x1 feature vector

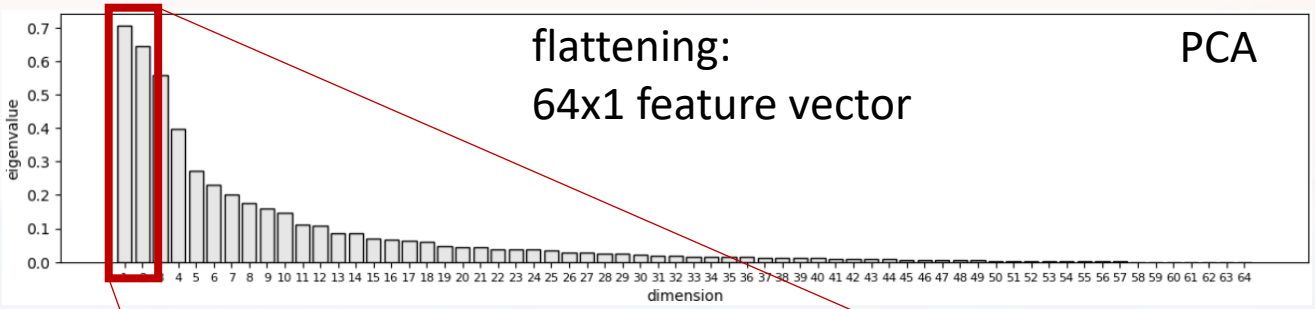
PCA



Advanced Visualization Tools: UMAP

digits data set

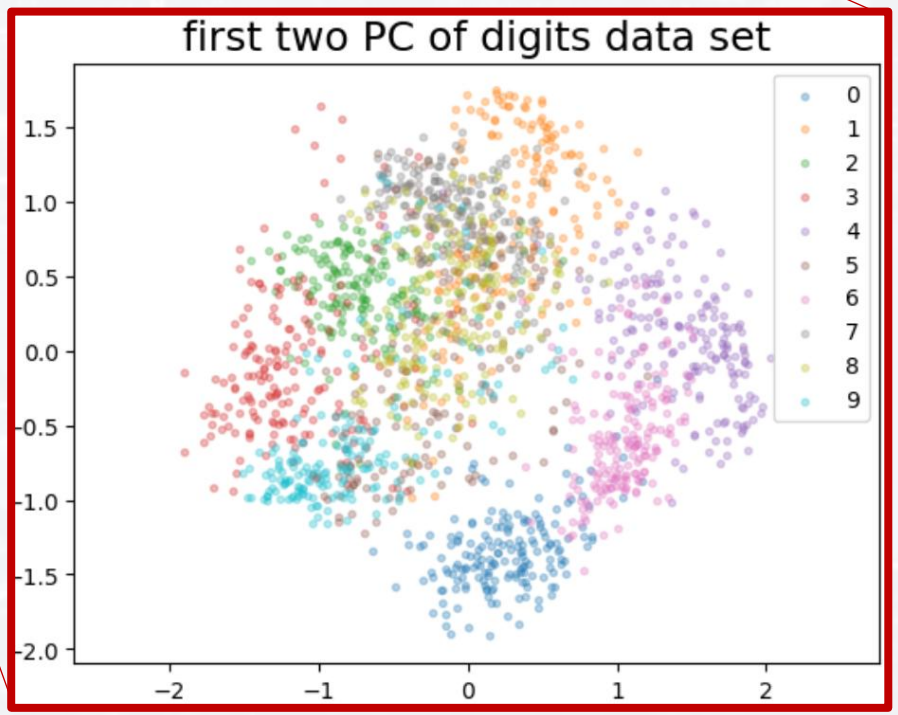




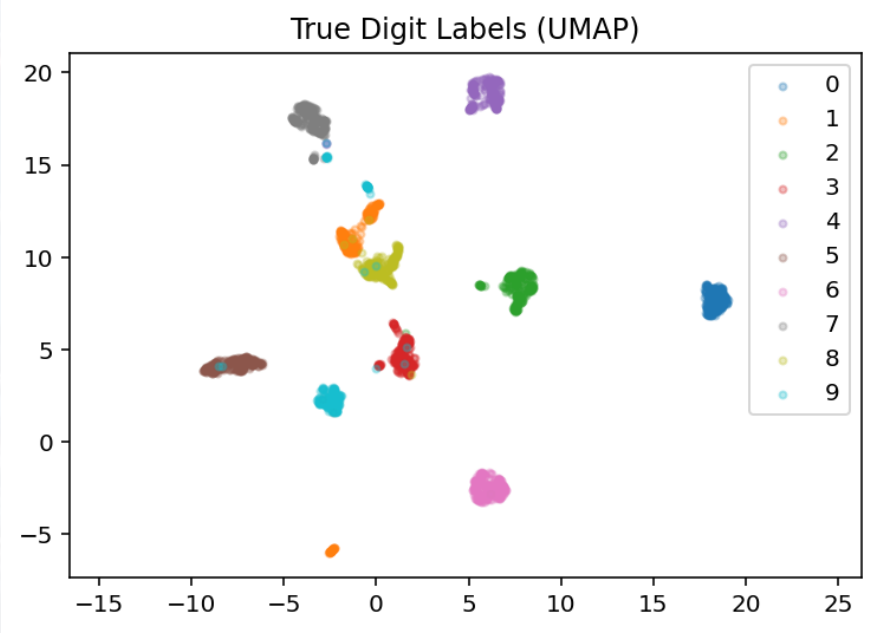
flattening:
64x1 feature vector

PCA

plotting first two PC (28% of variance)



Why does UMAP work so much better?



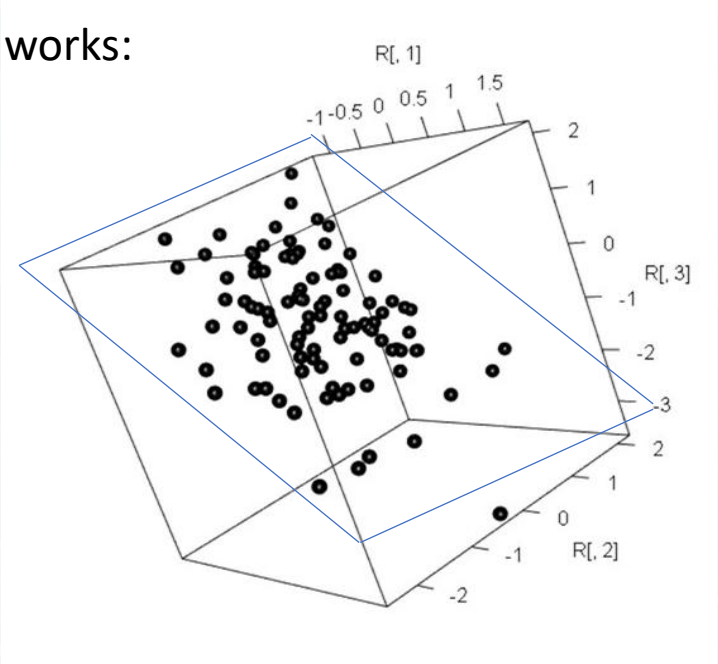
visualization in 2D or 3D is problematic:
big part of total variance is hidden in other directions

Advanced Visualization Tools: UMAP

PCA: + simple, straight forward

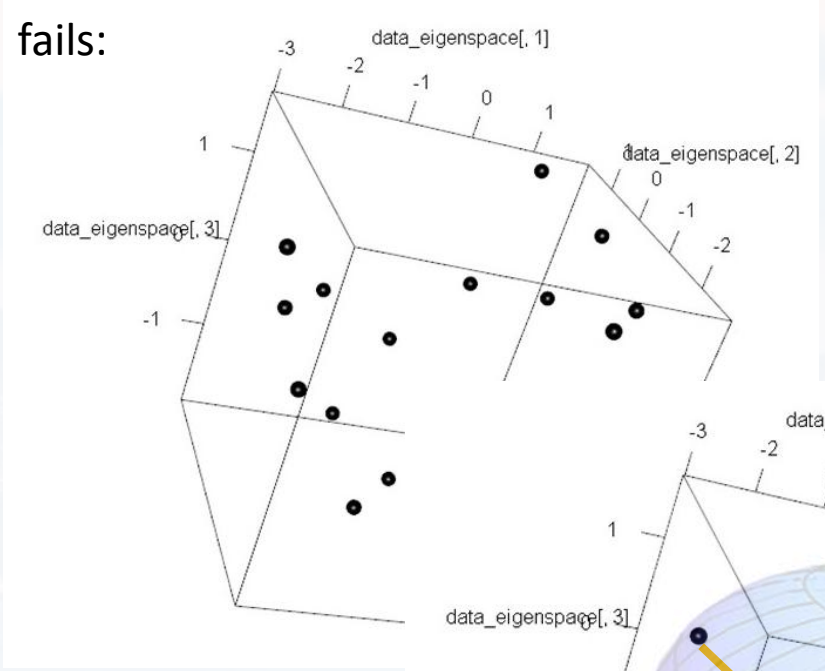
- data lives on curved manifold

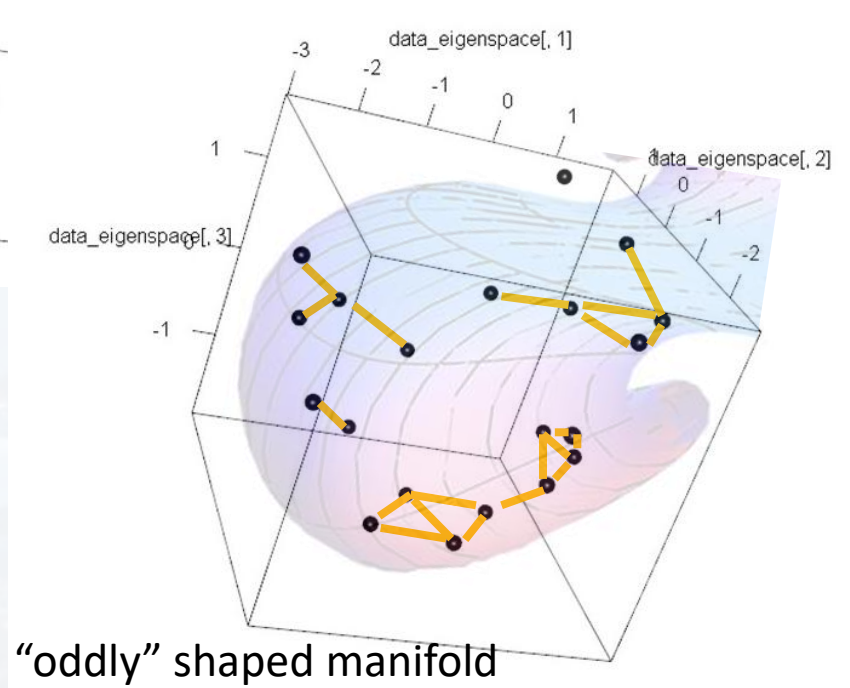
works:



flat **(means lin transformation possible)** manifold

fails:



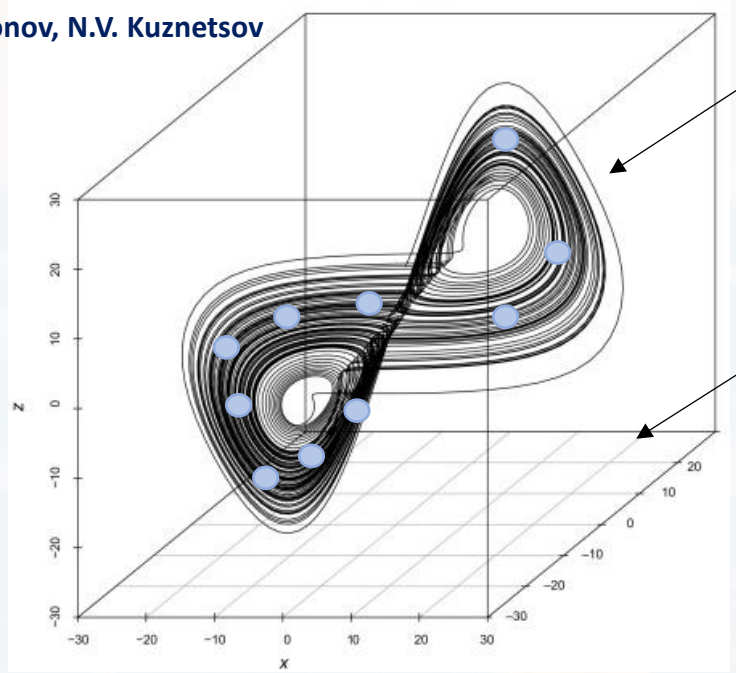


“oddly” shaped manifold

Uniform **M**anifold **A**pproximation and **P**rojection

Advanced Visualization Tools: UMAP

credit: G.A. Leonov, N.V. Kuznetsov



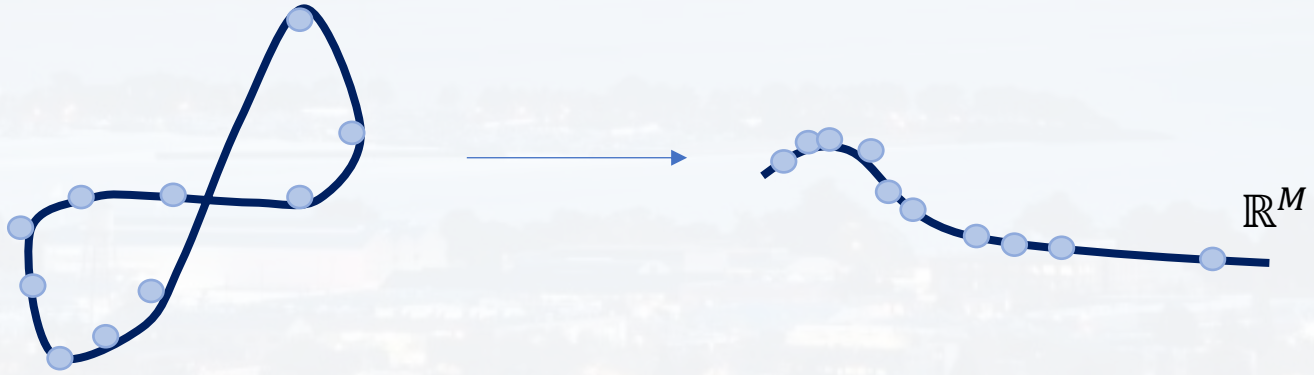
our data lives on a manifold $\mathcal{M}$
(on which it might cluster)

data points

$\mathcal{M}$ is embedded in high dimensional space $\mathbb{R}^N$

We only see the data in $\mathbb{R}^N$

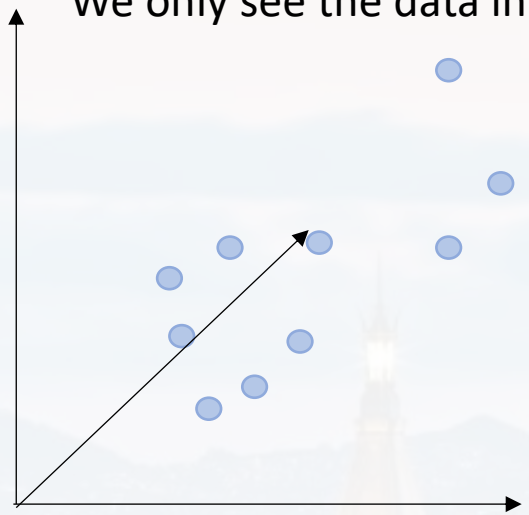
by approximating $\mathcal{M}$ we try to find a mapping to $\mathbb{R}^M$ with $M < N$



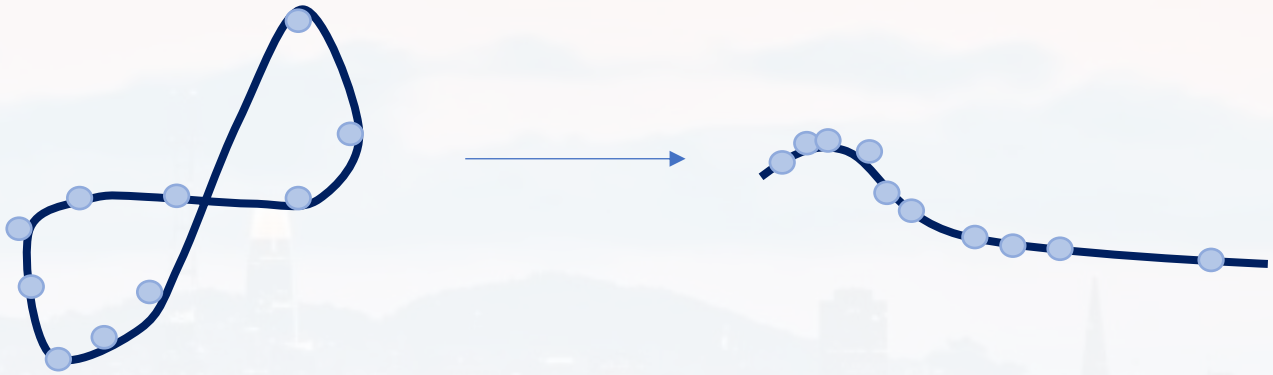
Advanced Visualization Tools: UMAP

data points

We only see the data in $\mathbb{R}^N$



by approximating $\mathcal{M}$ we try to find a mapping to $\mathbb{R}^M$ with $M < N$

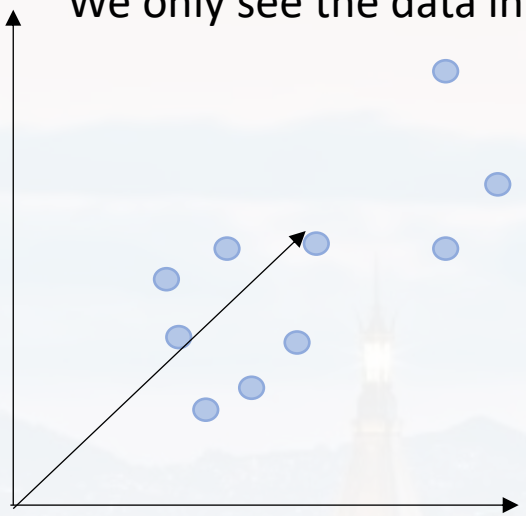


idea: reconstructing $\mathcal{M}$ from the data points in order to find a mapping to $\mathbb{R}^M$

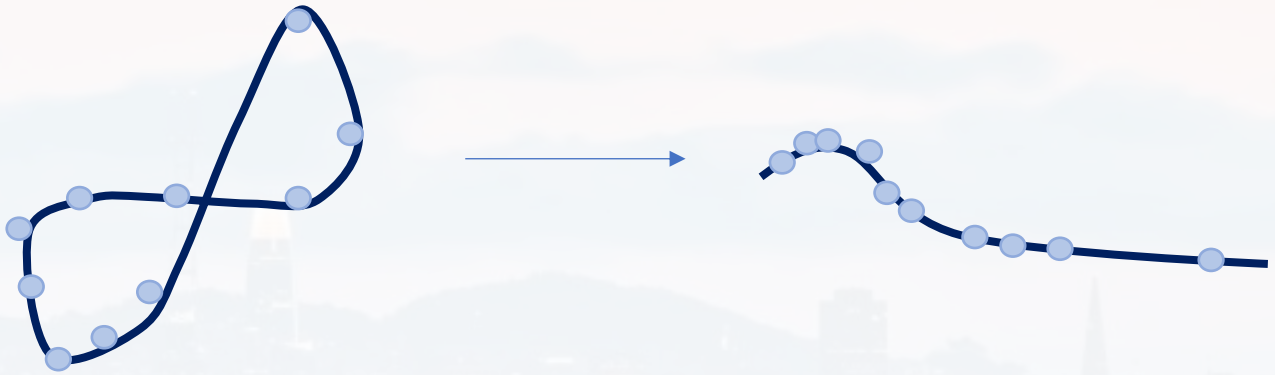
Advanced Visualization Tools: UMAP

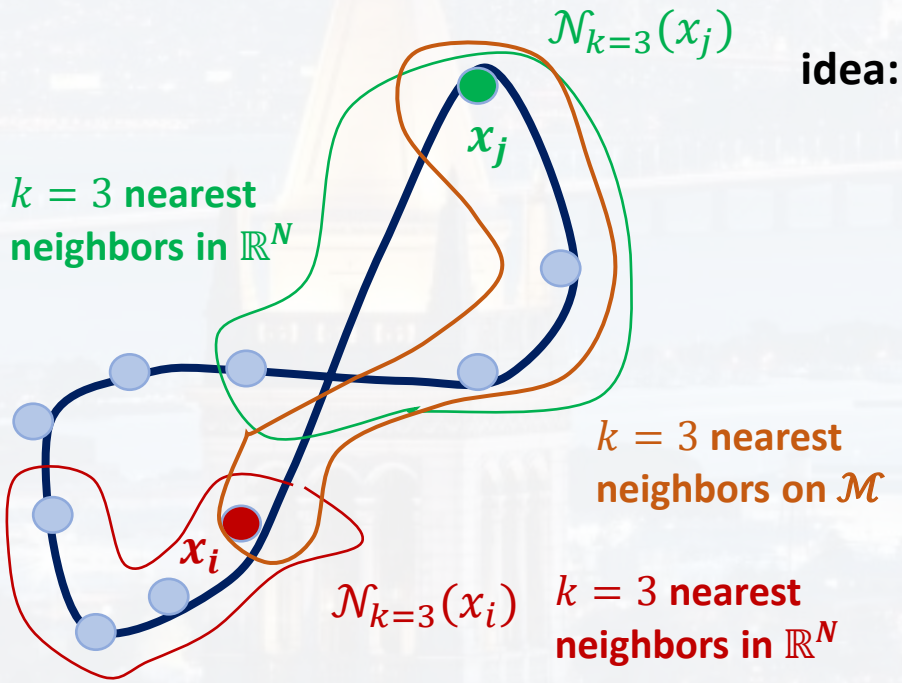
data points

We only see the data in $\mathbb{R}^N$



by approximating $\mathcal{M}$ we try to find a mapping to $\mathbb{R}^M$ with $M < N$





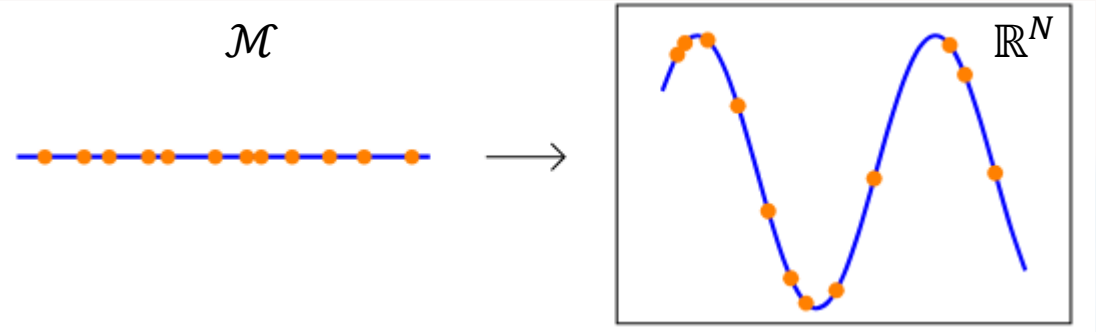
idea: reconstructing $\mathcal{M}$ from the data points in order to find a mapping to $\mathbb{R}^M$

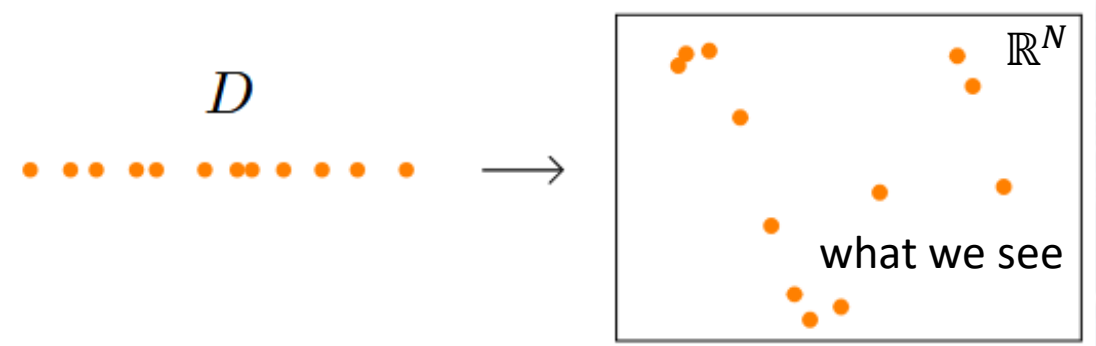
finding $\mathcal{N}_k(x_i)$ (**k-nearest neighbors**) in $\mathbb{R}^N$ of each data point x_i

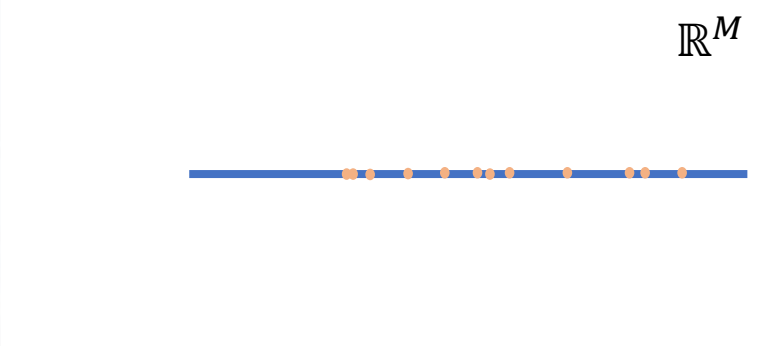
use these points in order to construct $\mathcal{M}$ assuming that the points where uniformly drawn from $\mathcal{M}$ (the "U" in UMAP)

Advanced Visualization Tools: UMAP

data points







idea: reconstructing $\mathcal{M}$ from the data points in order to find a mapping to $\mathbb{R}^M$

finding $\mathcal{N}_k(x_i)$ (**k-nearest neighbors**) in $\mathbb{R}^N$ of each data point x_i

use these points in order to construct $\mathcal{M}$ assuming that the points where uniformly drawn from $\mathcal{M}$ (the "U" in UMAP)

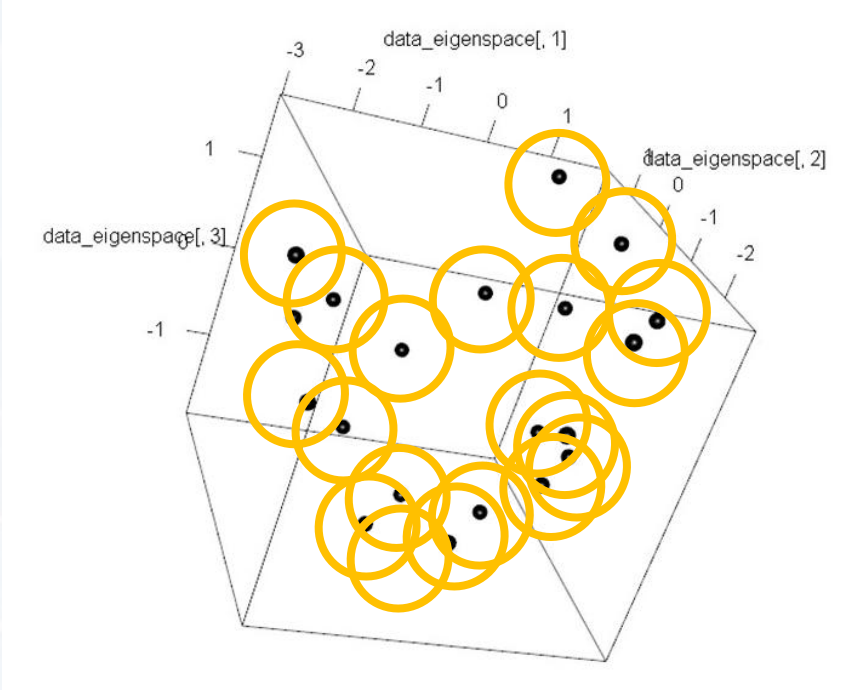
that does not mean that datapoints will be uniformly distributed in $\mathbb{R}^M$

note:
- need to account if k-th neighbor is an outlier
- k too large: averages the local structure of $\mathcal{M}$
- distances from $\mathcal{N}_k(x_i)$ for constructing $\mathcal{M}$ are actually no distances
  → some adjustments needed

figures see: Adele Jackson
"The mathematics of UMAP"

Advanced Visualization Tools:

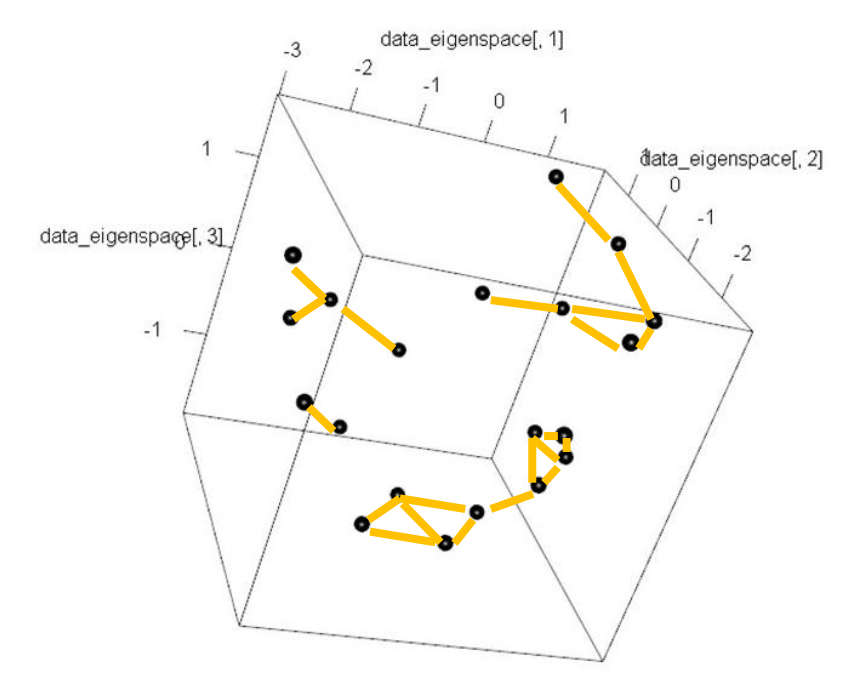
next step: connectivity



- defining a *unit* distance

Advanced Visualization Tools:

next step: connectivity



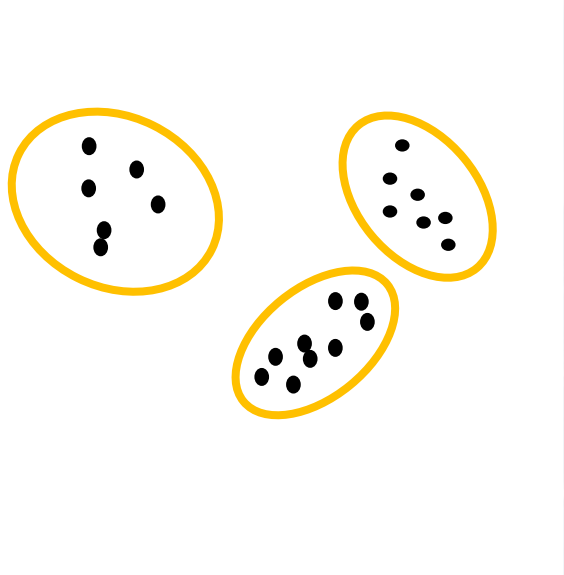
- defining a *unit* distance
- based on that *unit* distance

→ calculate probability $\mathcal{P}$ that data points are connected

→ $\mathcal{P} \sim \exp(-\textit{distance between two points})$

Advanced Visualization Tools: UMAP

next step: connectivity



- defining a *unit* distance
- based on that *unit* distance

  → calculate probability $\mathcal{P}$ that data points are connected

  → $\mathcal{P} \sim \exp(-\textit{distance between two points})$
- find a mapping to 2D or 3D

note:
- this was an extremely simplified explanation
- more details: Adele Jackson *"The mathematics of UMAP"* **(see repo and bCourses)**

Advanced Visualization Tools: UMAP

Python:

```
#pip install umap-learn
import umap.umap_ as umap
```

installing/importing UMAP

```
from sklearn.datasets      import load_digits
from sklearn.preprocessing import MinMaxScaler
```

loading the digits data set

```
digits  = load_digits()
XDigits = digits.data
YDigits = digits.target
```

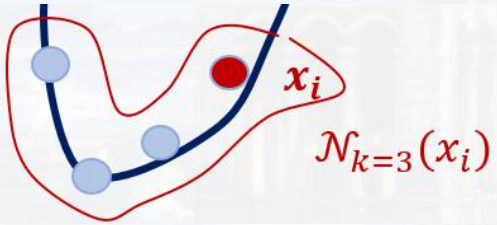
extracting data

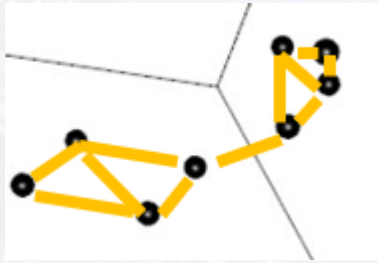
```
MinMax      = MinMaxScaler()
XSDigits    = MinMax.fit_transform(XDigits)
```

scaling data

```
newXYDigits = umap.UMAP().fit_transform(XSDigits)
```

actual UMAP





```
umap.UMAP(
UMAP(n_neighbors=15, n_components=2,
     metric='euclidean', metric_kwds=None,
     output_metric='euclidean',
     output_metric_kwds=None, n_epochs=None,
```

```
U = umap.UMAP(n_neighbors  = n_neighbors,  min_dist = min_dist,
              n_components = n_components, metric   = metric)
```

Advanced Visualization Tools: UMAP

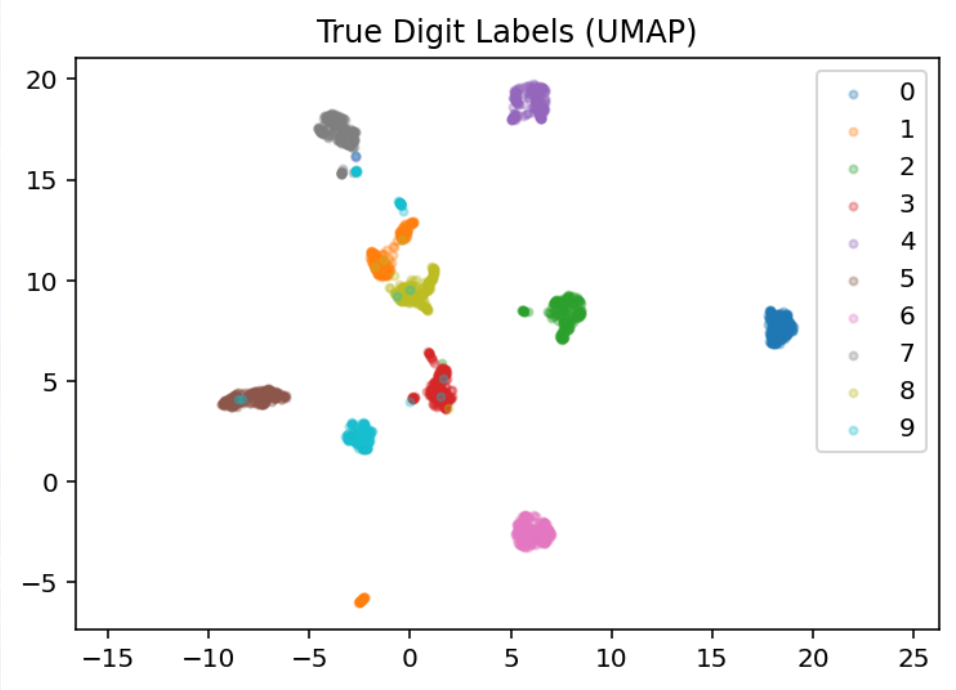
Python:

```
XDigits = digits.data
YDigits = digits.target

newXYDigits = umap.UMAP().fit_transform(XSDigits)

for i in range(np.max(YDigits) + 1):

        idx = np.argwhere(YDigits == i)
        plt.scatter(newXYDigits[idx, 0], newXYDigits[idx, 1], marker = '.',
                                                alpha = 0.3, label = str(i))

plt.gca().set_aspect('equal', 'datalim')
plt.title('UMAP projection of digits', fontsize = 18)
plt.legend()
plt.show()
```

Advanced Visualization Tools: UMAP

Python:

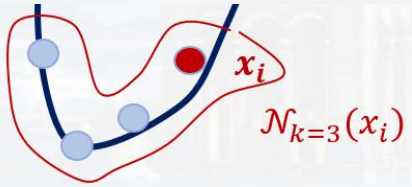
```
XDigits = digits.data
YDigits = digits.target

newXYDigits = umap.UMAP().fit_transform(XSDigits)

for i in range(np.max(YDigits) + 1):

        idx = np.argwhere(YDigits == i)
        plt.scatter(newXYDigits[idx, 0], newXYDigits[idx, 1], marker = '.',
                                                alpha = 0.3, label = str(i))

plt.gca().set_aspect('equal', 'datalim')
plt.title('UMAP projection of digits', fontsize = 18)
plt.legend()
plt.show()
```

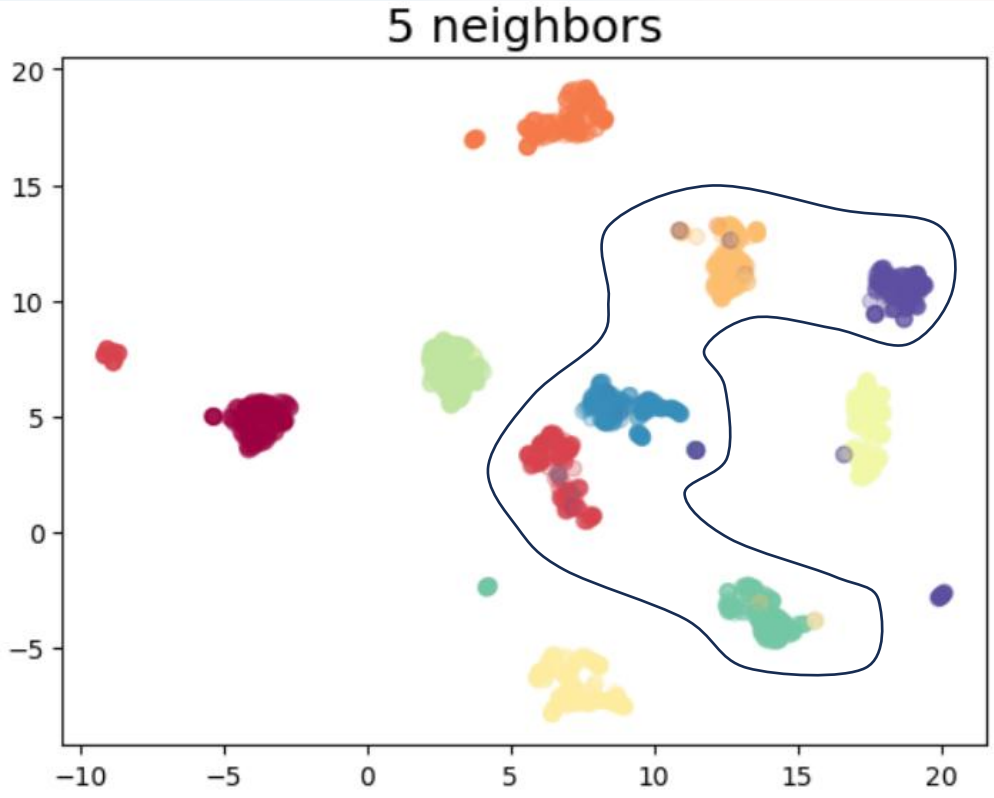


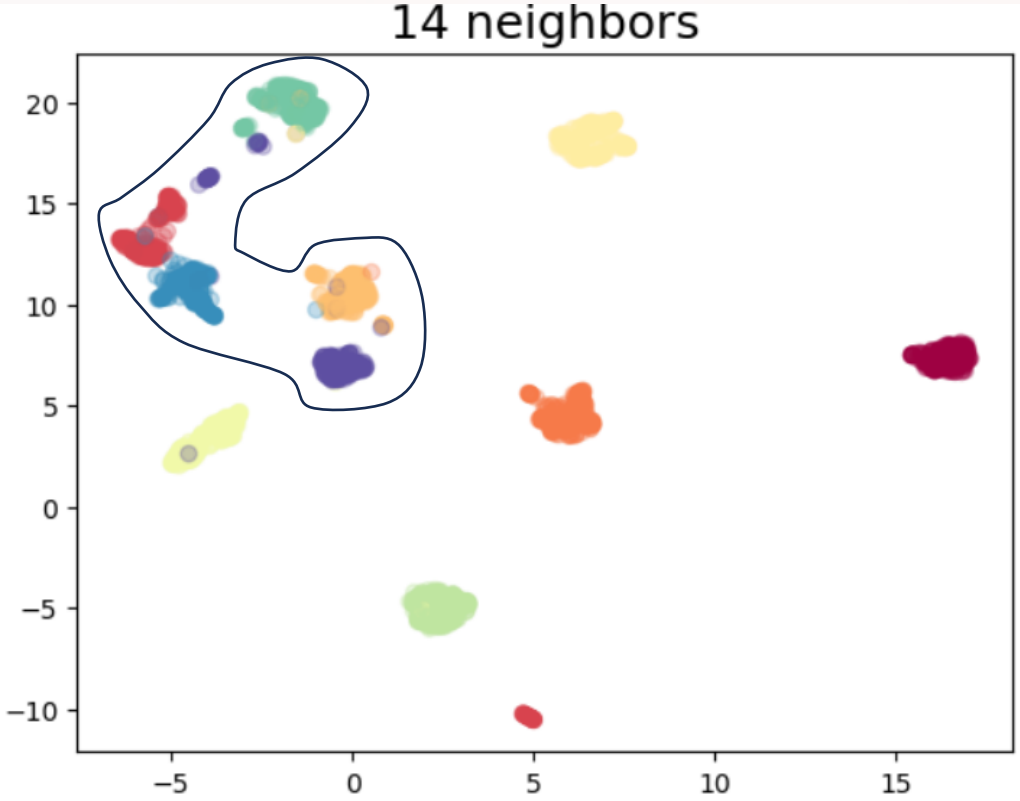
Advanced Visualization Tools: UMAP

Python:

```
U = umap.UMAP(n_neighbors  = n_neighbors,  min_dist = min_dist,
              n_components = n_components, metric   = metric)
```







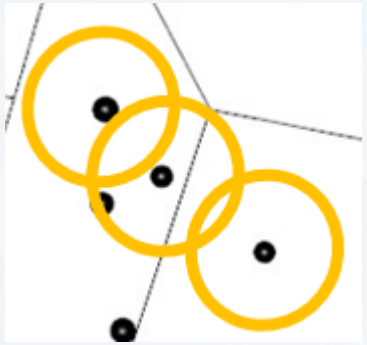
note that some cluster locations are flipped: there is no natural up/down or left/right!

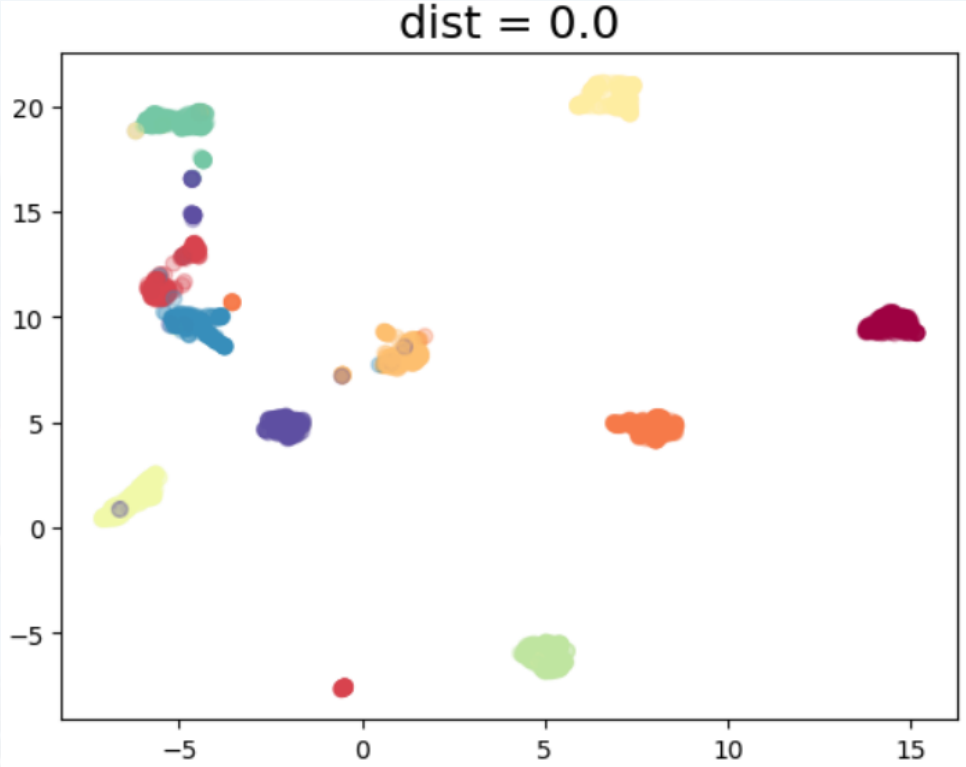
larger k (more neighbors): better statistics for approximating $\mathcal{M}$, but averages out finer features of $\mathcal{M}$
...and vice verse

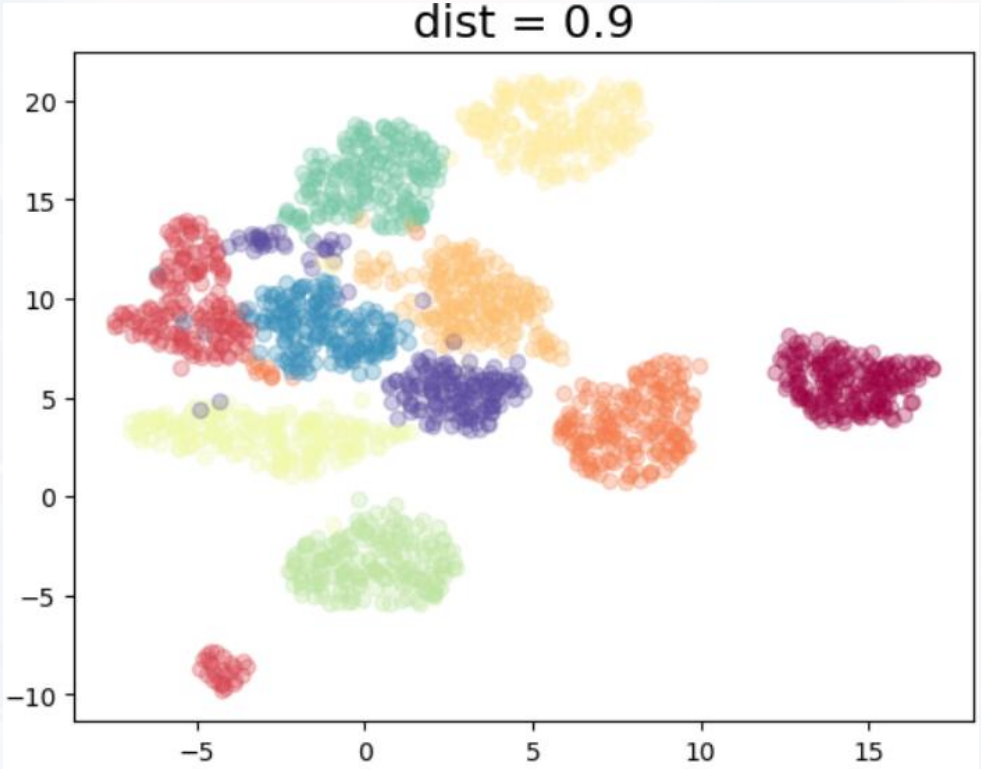
Advanced Visualization Tools: UMAP

Python:

```
U = umap.UMAP(n_neighbors  = n_neighbors,  min_dist = min_dist,
              n_components = n_components, metric   = metric)
```







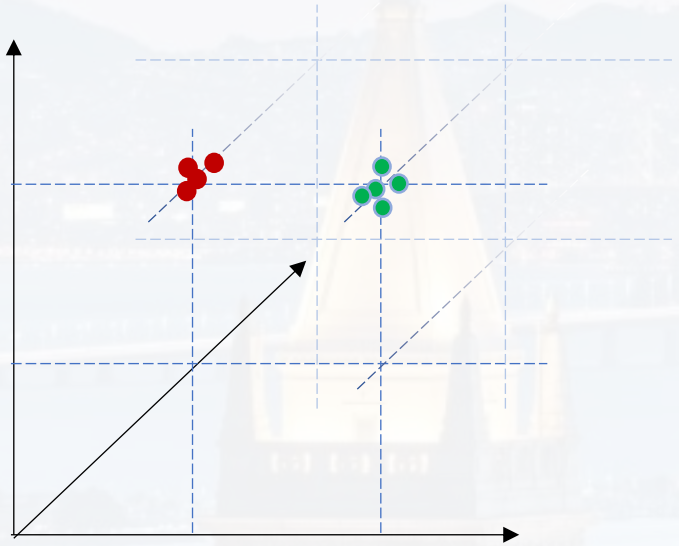
larger the distance, the more data points are connected, the “fuzzier” the cluster get

Advanced Visualization Tools: UMAP

Python:

```
U = umap.UMAP(n_neighbors  = n_neighbors,  min_dist = min_dist,
              n_components = n_components, metric   = metric)
```

note: clusters can appear “artificially”, if many features are discrete (like in the Alzheimer data set)



Let’s explore `WalkThroughUMAP.ipynb`

Advanced Visualization Tools

credit: David C. Lovelace

Outline

UMAP

t-SNE

Leiden

DBSCAN

Advanced Visualization Tools: t-SNE

t-distributed **S**tochastic **N**eighbor **E**mbedding

- similar basic idea like UMAP (mapping data from $\mathbb{R}^N$ to $\mathbb{R}^M$ where $M < N$)

- in $\mathbb{R}^N$: probability $\mathcal{P}$ that two data points x_i and x_j are neighbors:

$$\mathcal{P}(j|i) \sim \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)$$

σ_i^2 influences how far data points can be in order to considered neighbors (similar effect in choosing k in UMAP); indirectly determined by input argument **"perplexity"**

- this will lead to the **t-distribution** when calculating the joint probability $\mathcal{P}(j,i) \sim \frac{\mathcal{P}(j|i)+\mathcal{P}(i|j)}{2}$

- mapping to $\mathbb{R}^M$ by keeping spatial distribution in $\mathbb{R}^M$ as close as possible to the one in $\mathbb{R}^N$ by minimizing the **KL-divergence** (see Module 3)

- this is done via **gradient descent** (see Module 13)

see Journal of Machine Learning Research 9 (2008) 2579-2605

Advanced Visualization Tools: t-SNE

```
from sklearn.manifold import TSNE
```

importing t-SNE

```
from sklearn.datasets      import load_digits
from sklearn.preprocessing import MinMaxScaler

digits  = load_digits()
XDigits = digits.data
YDigits = digits.target

MinMax      = MinMaxScaler()
XSDigits    = MinMax.fit_transform(XDigits)
```

values between 5 – 50 recommended, should not exceed number of data points

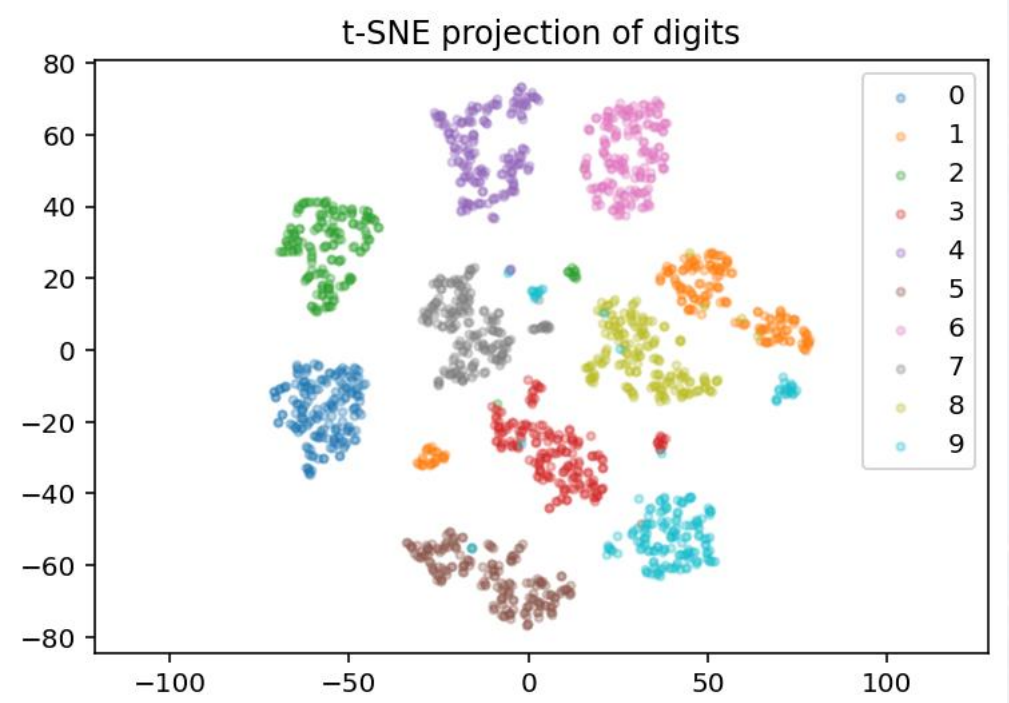
```
TSNE(
    TSNE(n_components=2, *, perplexity=30.0,
         early_exaggeration=12.0, learning_rate='auto',
         max_iter=1000, n_iter_without_progress=300,
         min_grad_norm=1e-07, metric='euclidean',
```

```
newXYDigits = TSNE(n_components = 2, learning_rate = 'auto',
                   init = 'random', perplexity = 10).fit_transform(XSDigits)
```

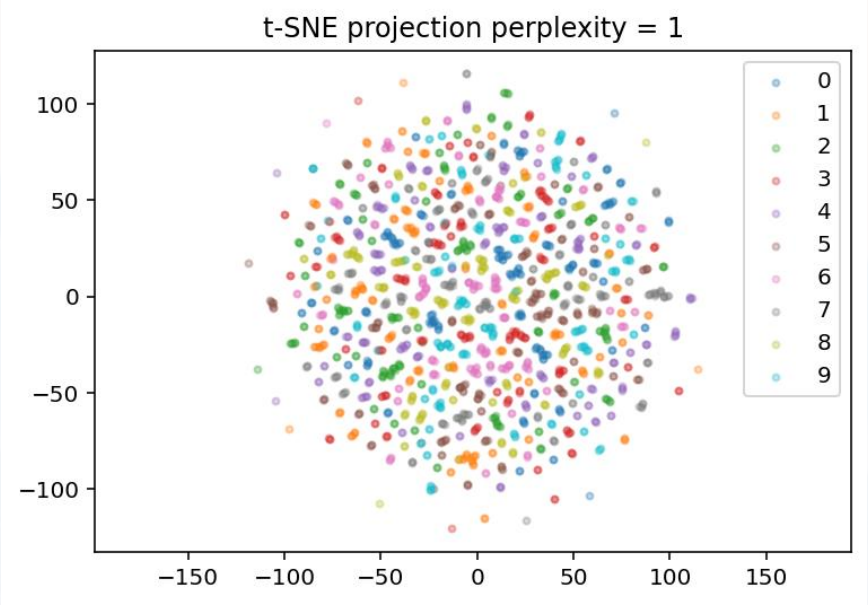
random initialization of embedding (alternatively: *'pca'*)

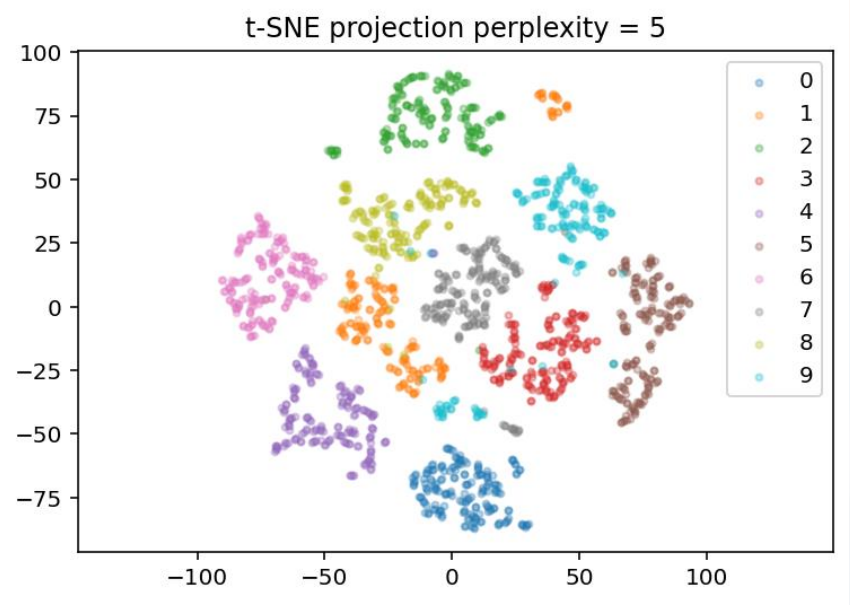
Advanced Visualization Tools: t-SNE

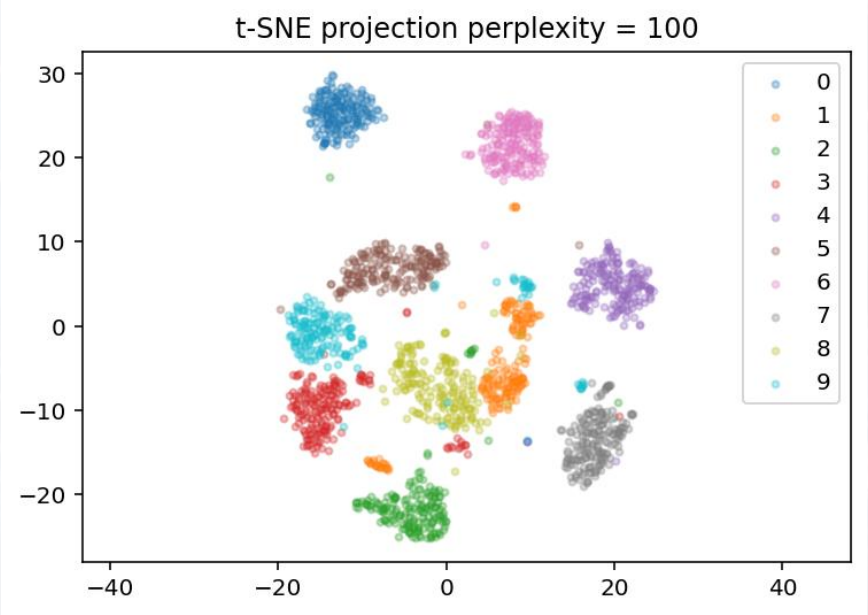
```
for i in range(np.max(YDigits) + 1):

    idx = np.argwhere(YDigits == i)
    plt.scatter(newXYDigits[idx, 0], newXYDigits[idx, 1], marker = '.',
                alpha = 0.3, label = str(i))

plt.gca().set_aspect('equal', 'datalim')
plt.title('t-SNE projection of digits', fontsize = 12)
plt.legend()
plt.show()
```

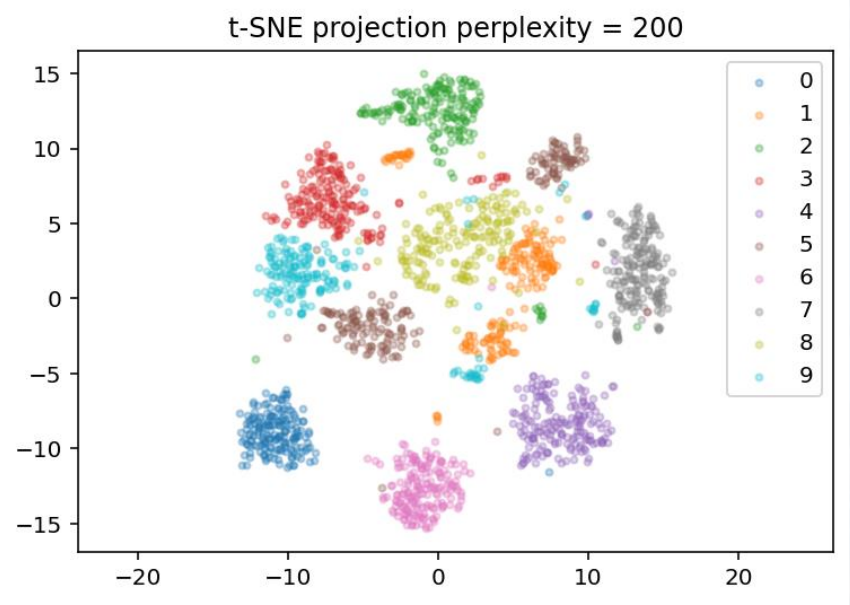


Advanced Visualization Tools: t-SNE





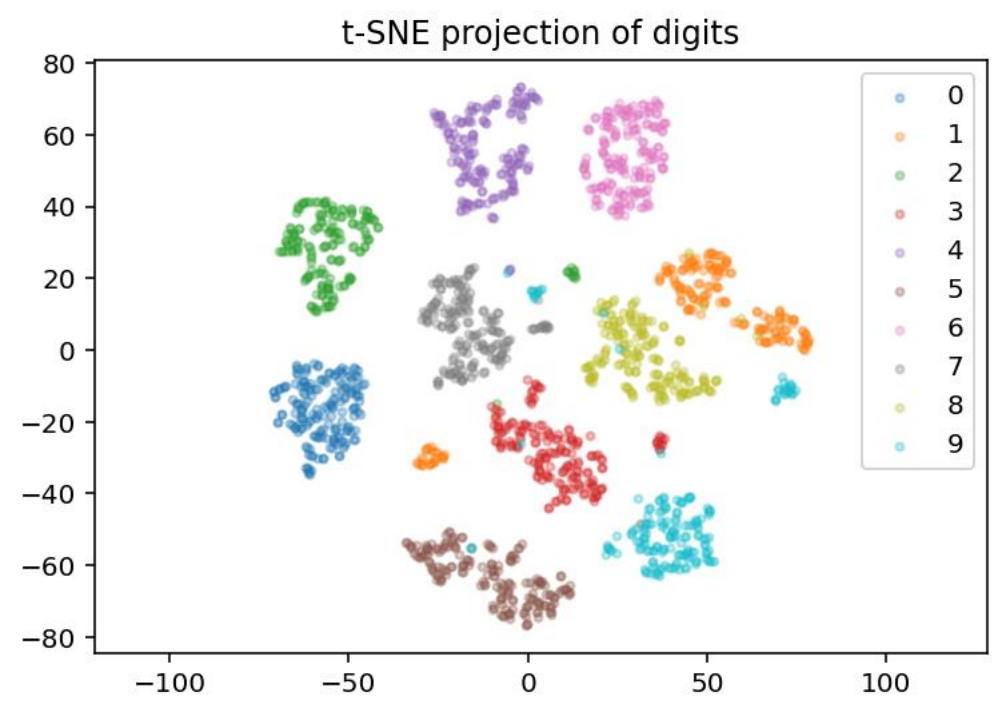




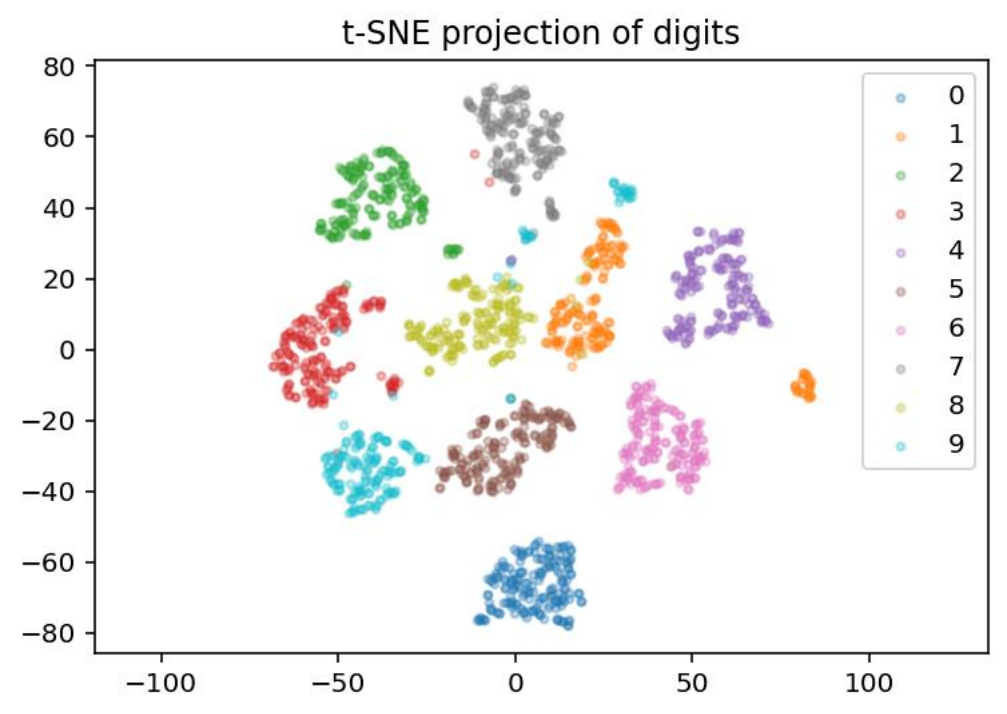
Advanced Visualization Tools: t-SNE

```
perplexity = 10
```

```
init = 'random'
```




```
init = 'pca'
```



Advanced Visualization Tools

credit: David C. Lovelace

Outline

UMAP

t-SNE

Leiden

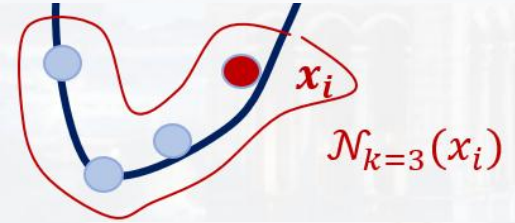
DBSCAN

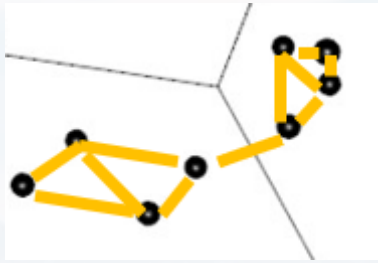
Advanced Visualization Tools:

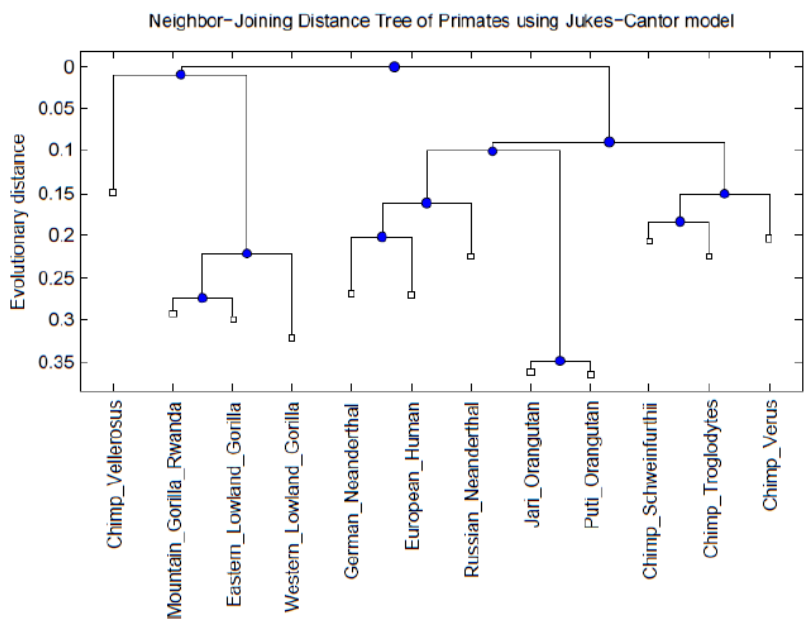
Leiden

UMAP & t-NSE work with connectivity

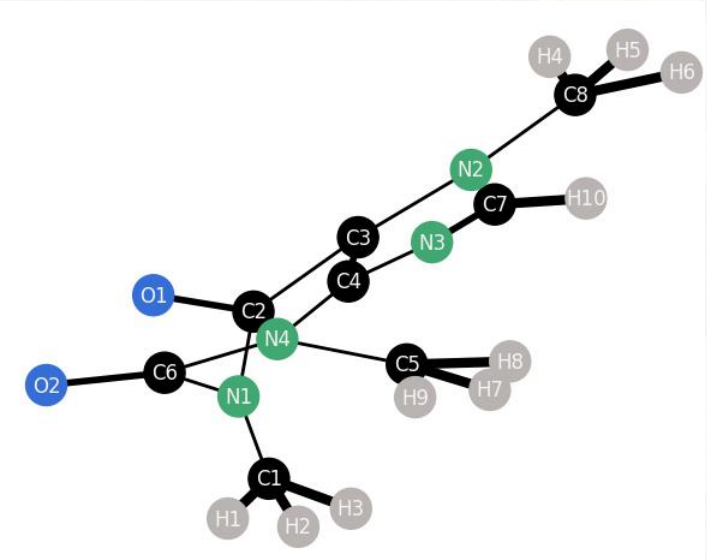
→ interpreting data clouds as *graphs*

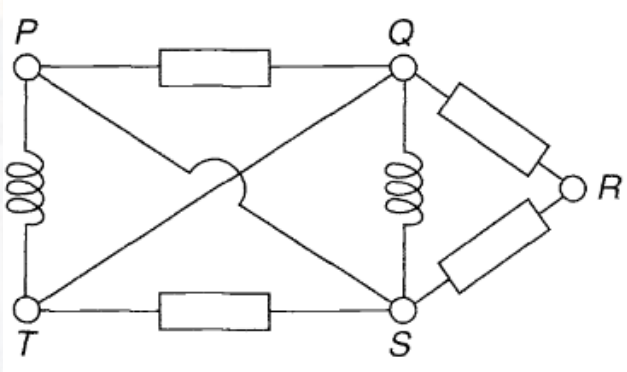


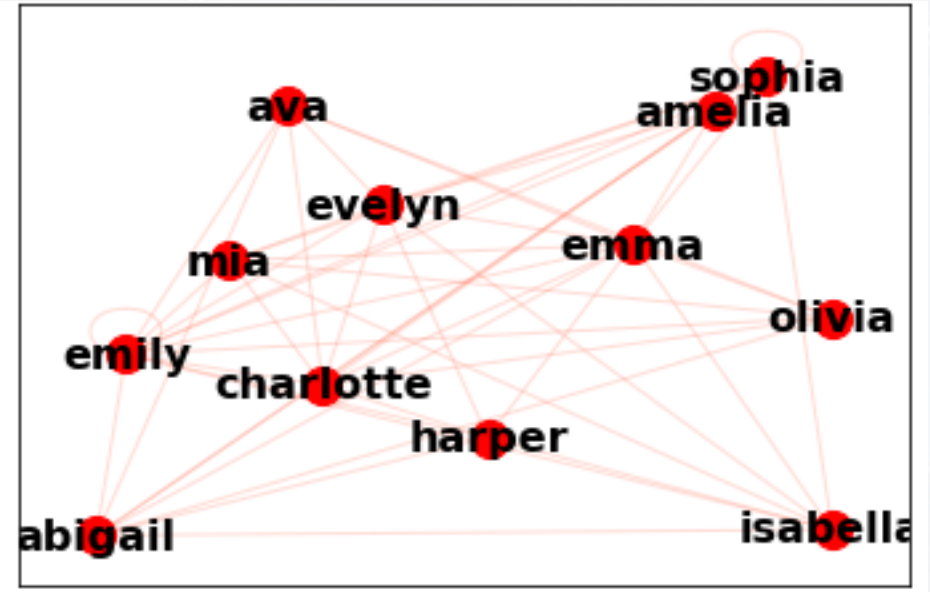




graphs:

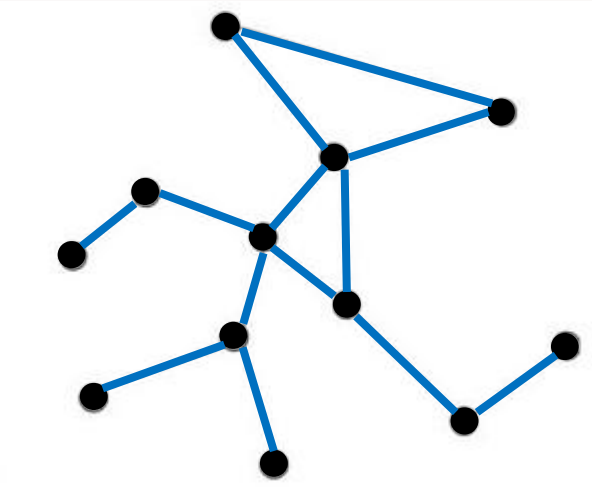






Advanced Visualization Tools:

graphs:



Graph G

nodes N (vertices V)

edges E

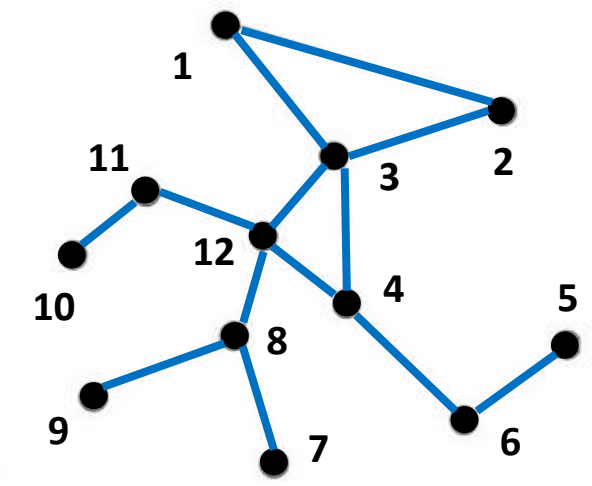
$G = G(N, E)$

- social networks
- street maps
- workflows/planning
- biological signal pathways
- image processing
- diffusion processes

- nodes can have **features**
  - molecules: mass/ electronegativity
  - people: age, income, sex, …
- edges can have **attributes**
  - molecules: bond length/strength
  - people: relations (work, friend, family)

Advanced Visualization Tools:

graphs:



structural information: **adjacency matrix A**

$A_{ij} = 1$ if $(n_i, n_j) \in E$

(nodes n_i and n_j have a common edge)

$A_{ij} = 0$ else

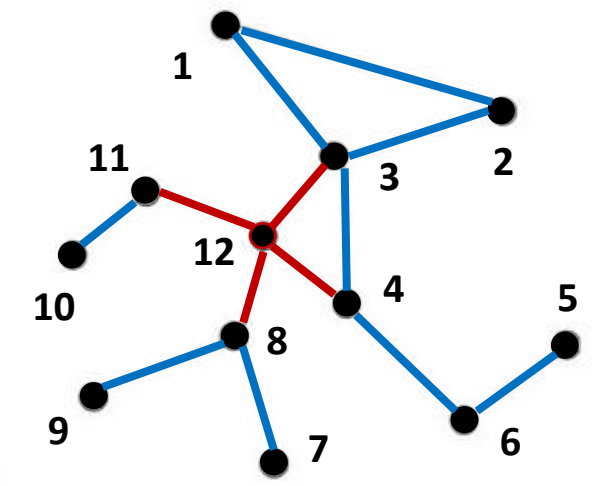
Graph $G = G(N, E)$

nodes N (vertices V)
edges E

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Advanced Visualization Tools:

graphs:



structural information: **adjacency matrix A**

$A_{ij} = 1$ if $(n_i, n_j) \in E$

(nodes n_i and n_j have a common edge)

$A_{ij} = 0$ else

Graph $G = G(N, E)$

nodes N (vertices V)
edges E

node 12 has four first degree neighbors

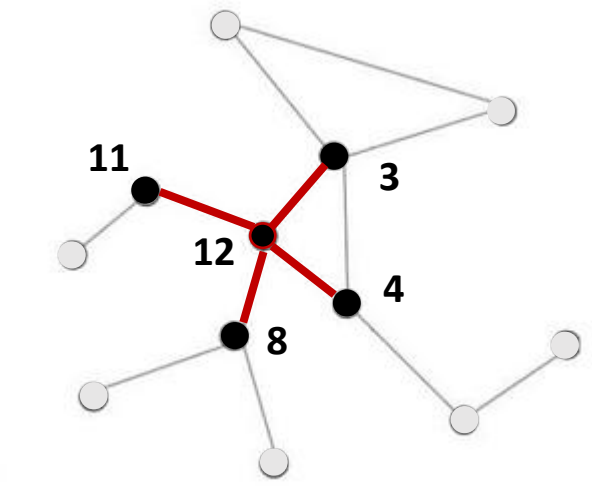
degree *d* of a node

$$d(n_i) = \sum_j A_{ij}$$

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & \mathbf{1} & \mathbf{1} & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & \mathbf{1} & 0 \end{pmatrix}$$

Advanced Visualization Tools:

graphs:



structural information: **adjacency matrix A**

$A_{ij} = 1$ if $(n_i, n_j) \in E$

(nodes n_i and n_j have a common edge)

$A_{ij} = 0$ else

Graph $G = G(N, E)$

nodes N (vertices V)
edges E

node 12 has four first degree neighbors

degree *d* of a node

$$d(n_i) = \sum_j A_{ij}$$

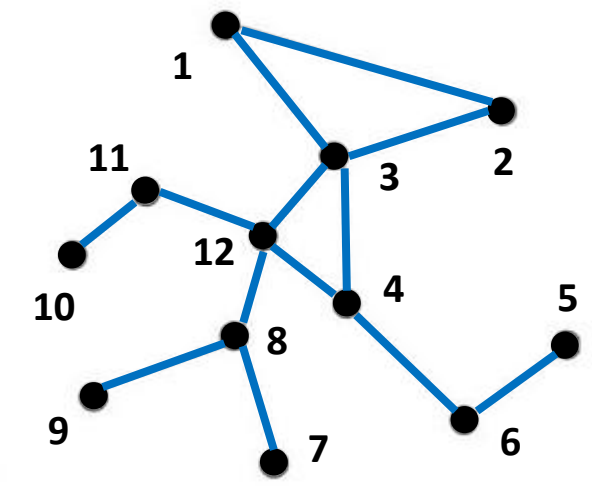
first degree neighborhood $\mathcal{N}$

$$\mathcal{N}(n_i) = \{n_j \in N : (n_i, n_j) \in E\}$$

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & \mathbf{1} & \mathbf{1} & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & \mathbf{1} & 0 \end{pmatrix}$$

Advanced Visualization Tools:

graphs:



structural information: **adjacency matrix A**

$A_{ij} = 1$ if $(n_i, n_j) \in E$

(nodes n_i and n_j have a common edge)

$A_{ij} = 0$ else

Graph $G = G(N, E)$

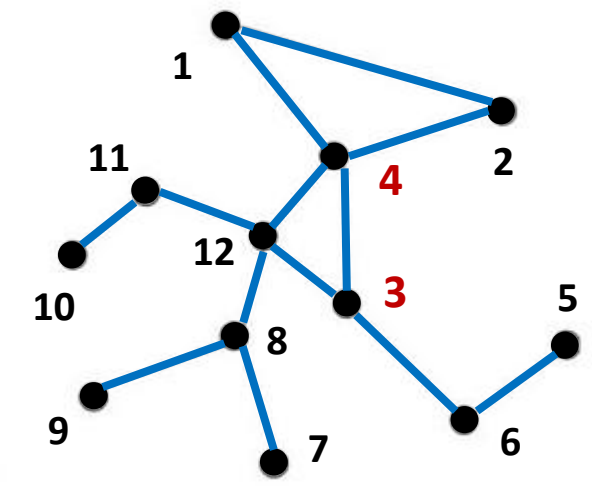
nodes N (vertices V)
edges E

The order of enumerating the nodes is not relevant!
(**permutation invariance**)

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Advanced Visualization Tools:

graphs:



structural information: **adjacency matrix A**

$A_{ij} = 1$ if $(n_i, n_j) \in E$

(nodes n_i and n_j have a common edge)

$A_{ij} = 0$ else

Graph $G = G(N, E)$

nodes N (vertices V)
edges E

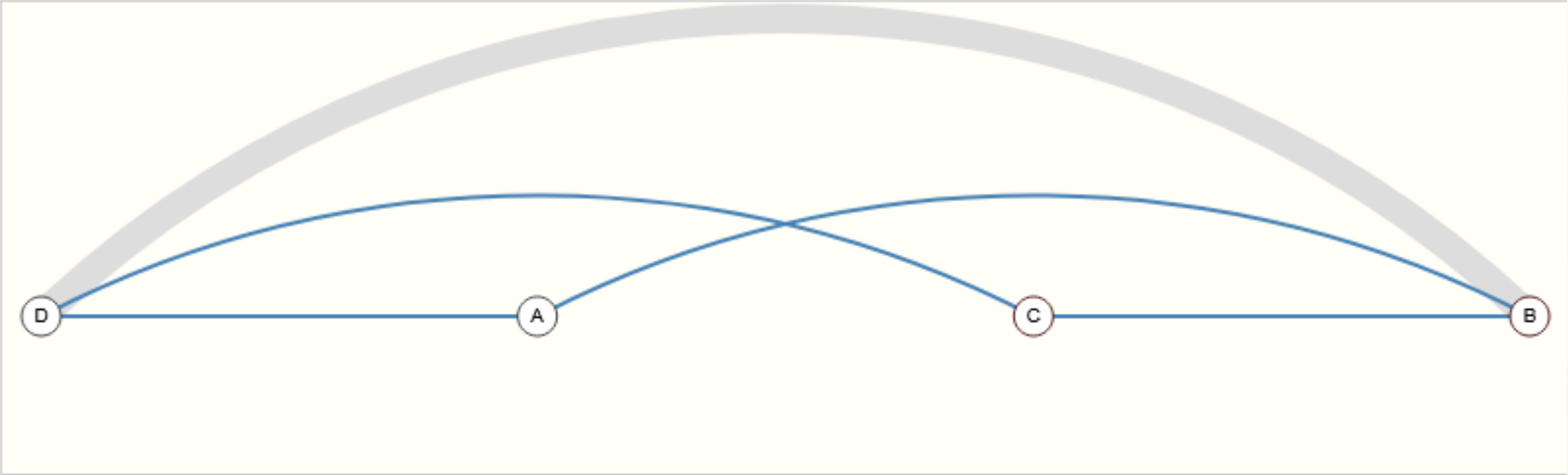
The order of enumerating the nodes is not relevant!
(**permutation invariance**)

Each graph can be represented by **N!** adjacency matrices!

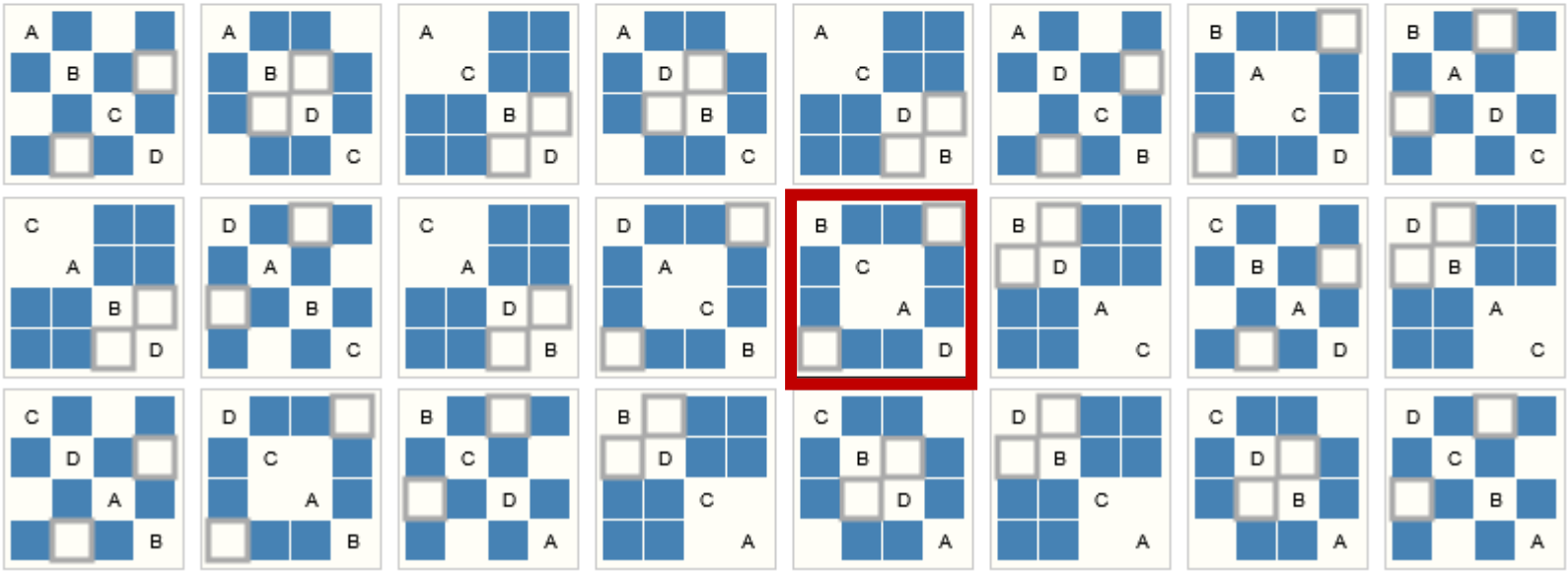
$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Advanced Visualization Tools:

graphs: Each graph can be represented by **N!** adjacency matrices!



[animation here]

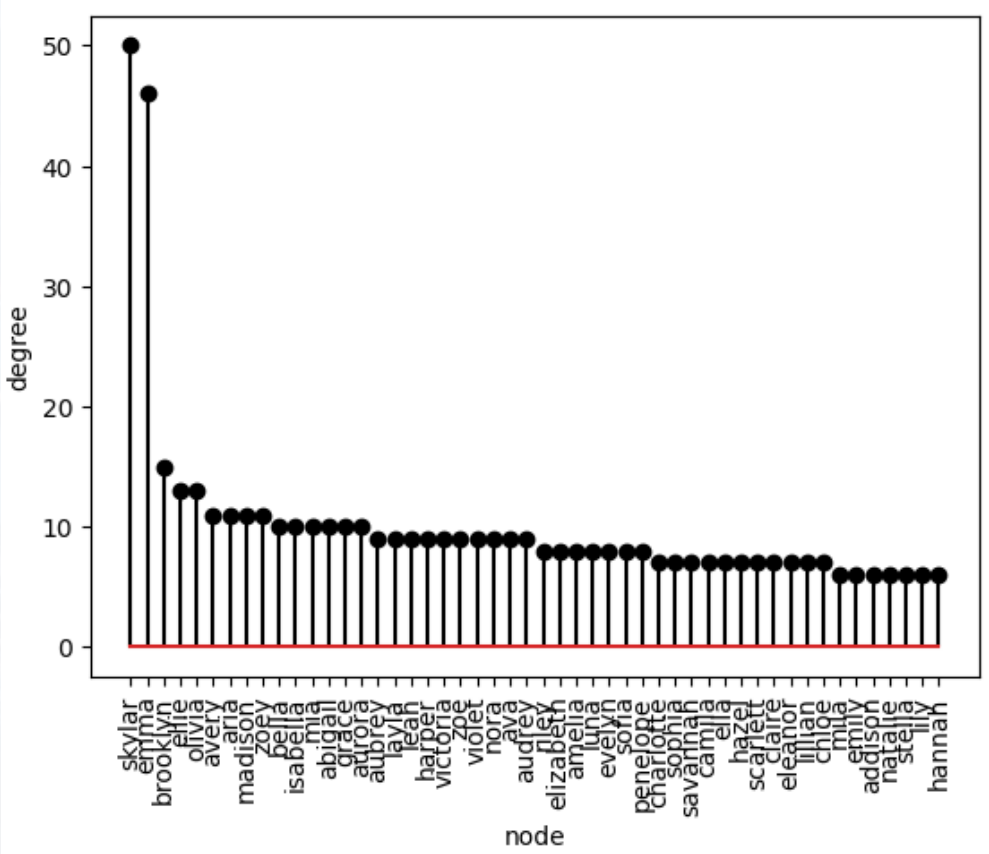


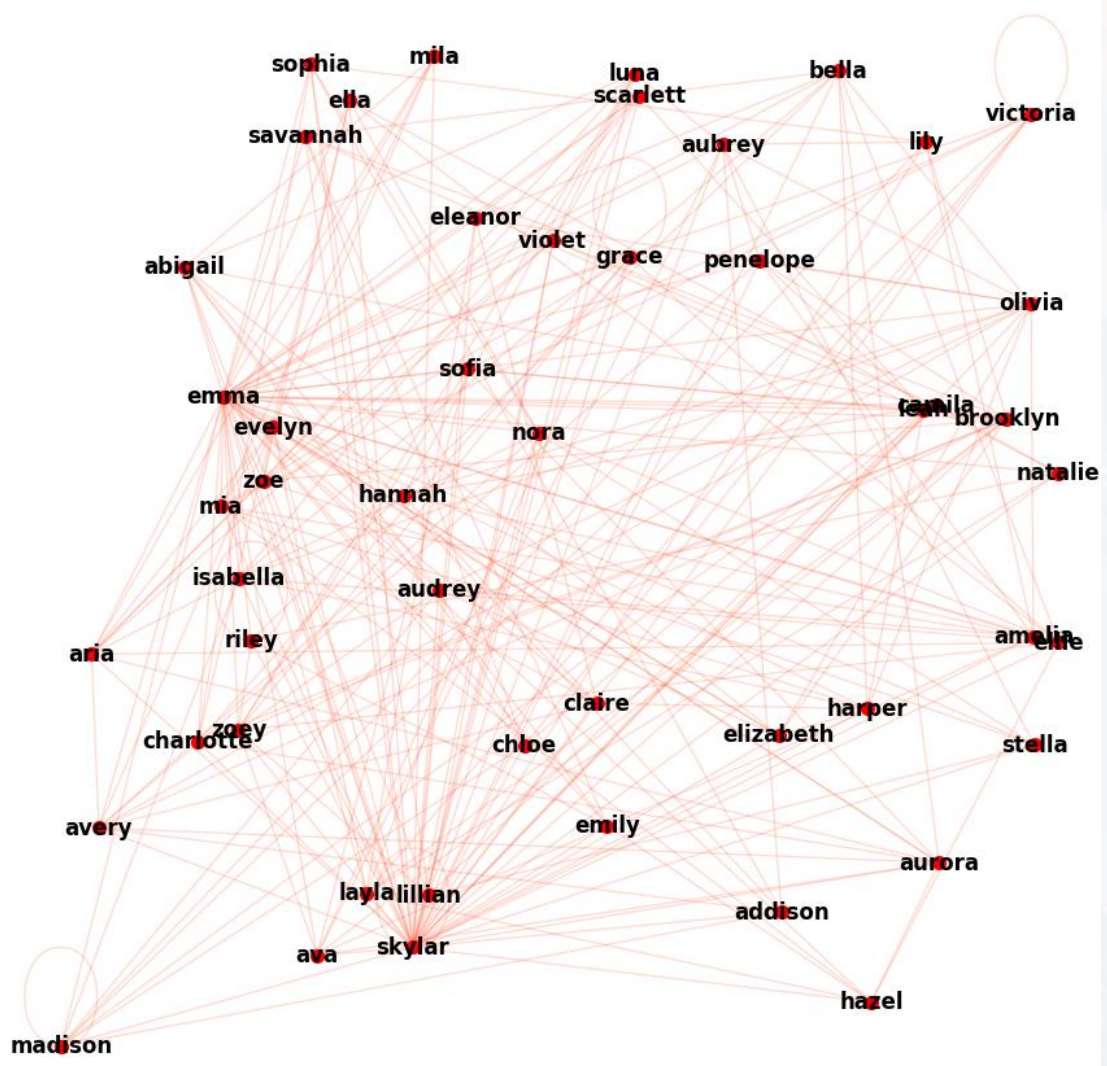
Advanced Visualization Tools:

graphs: visualizing a graph:

```
import networkx as nx #pip install networkx
```

see: `Graph_I.ipynb`





Advanced Visualization Tools:

graphs:

$$d(n_i) = \sum_j A_{ij}$$

degree of node n_i

$$\mathcal{N}(n_i) = \{n_j \in N: (n_i, n_j) \in E\}$$

neighborhood $\mathcal{N}(n_i)$ of node n_i
for first degree neighborhood $|\mathcal{N}(n_i)| = d(n_i)$

$$S_{com} = |\mathcal{N}(n_i) \cap \mathcal{N}(n_j)|$$

number for neighbors nodes n_i and n_j have in common.

idea: nodes with many common neighbors are more likely to be similar or have a potential connection **(circle of friends)**.

$$S_{rat} = \frac{|\mathcal{N}(n_i) \cap \mathcal{N}(n_j)|}{|\mathcal{N}(n_i) \cup \mathcal{N}(n_j)|}$$

ratio for neighbors nodes n_i and n_j have in common.

note:
There are more quantities ("importance", "centrality" etc.), but they are all a function of A_{ij}.

Advanced Visualization Tools: Leiden

UMAP & t-NSE work with connectivity

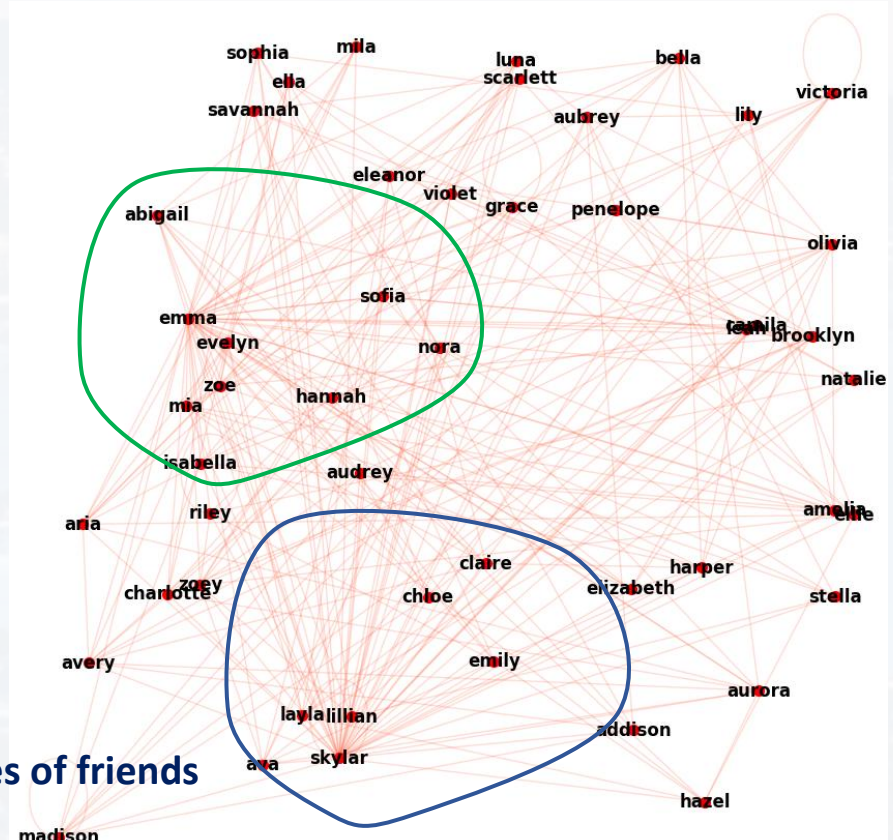
$$d(n_i) = \sum_j A_{ij}$$

→ interpreting data clouds as *graphs*

- connecting each datapoint to its k nearest neighbors

- calculate graph **partitions** from these connections
  → **communities** (= set of nodes, like a **circle of friends**)

note: Nodes can belong to only *one* community. People can either be friends with Skylar *or* Evelyn!

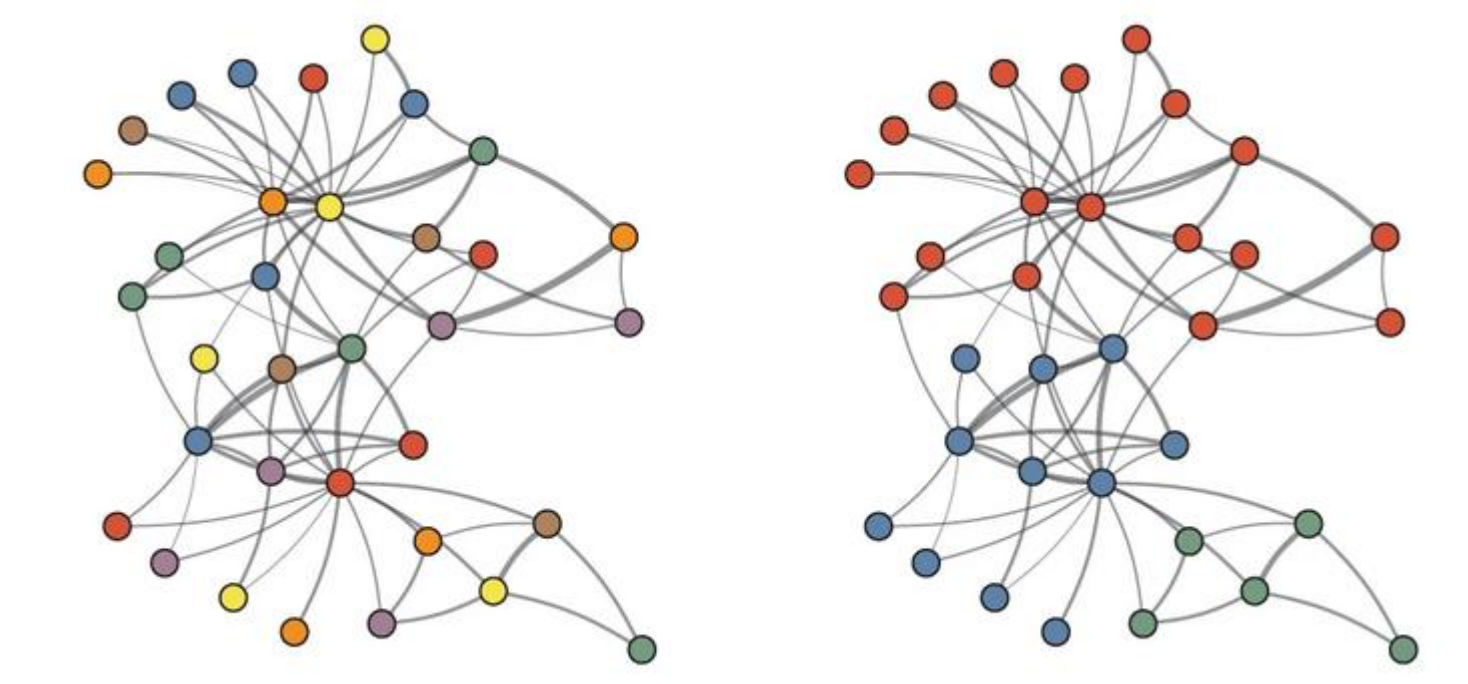


Advanced Visualization Tools: Leiden

- connecting each datapoint to its k nearest neighbors

$$d(n_i) = \sum_j A_{ij}$$

- calculate graph **partitions** from these connections
  → **communities** (= set of nodes, like a **circle of friends**)



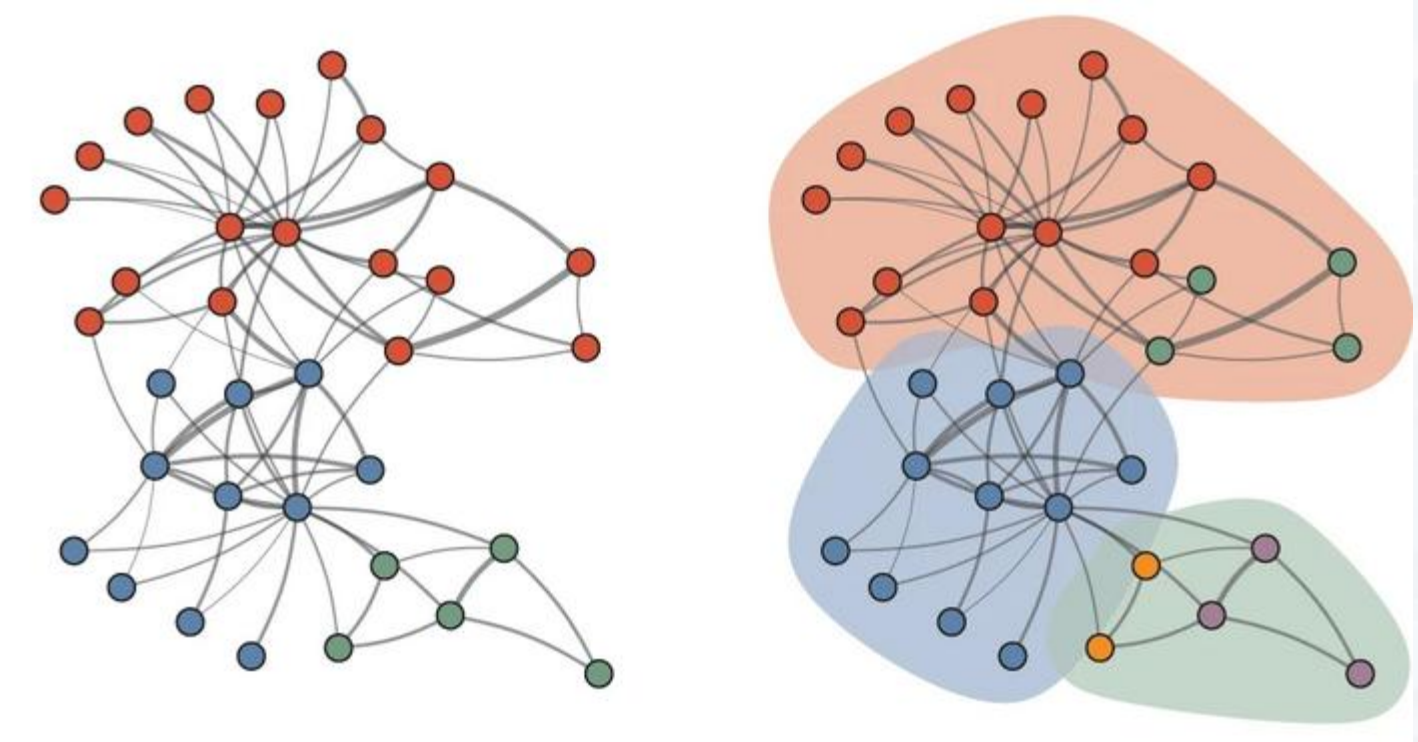
source

Advanced Visualization Tools: Leiden

- connecting each datapoint to its k nearest neighbors

$$d(n_i) = \sum_j A_{ij}$$

- calculate graph **partitions** from these connections
  → **communities** (= set of nodes, like a **circle of friends**)



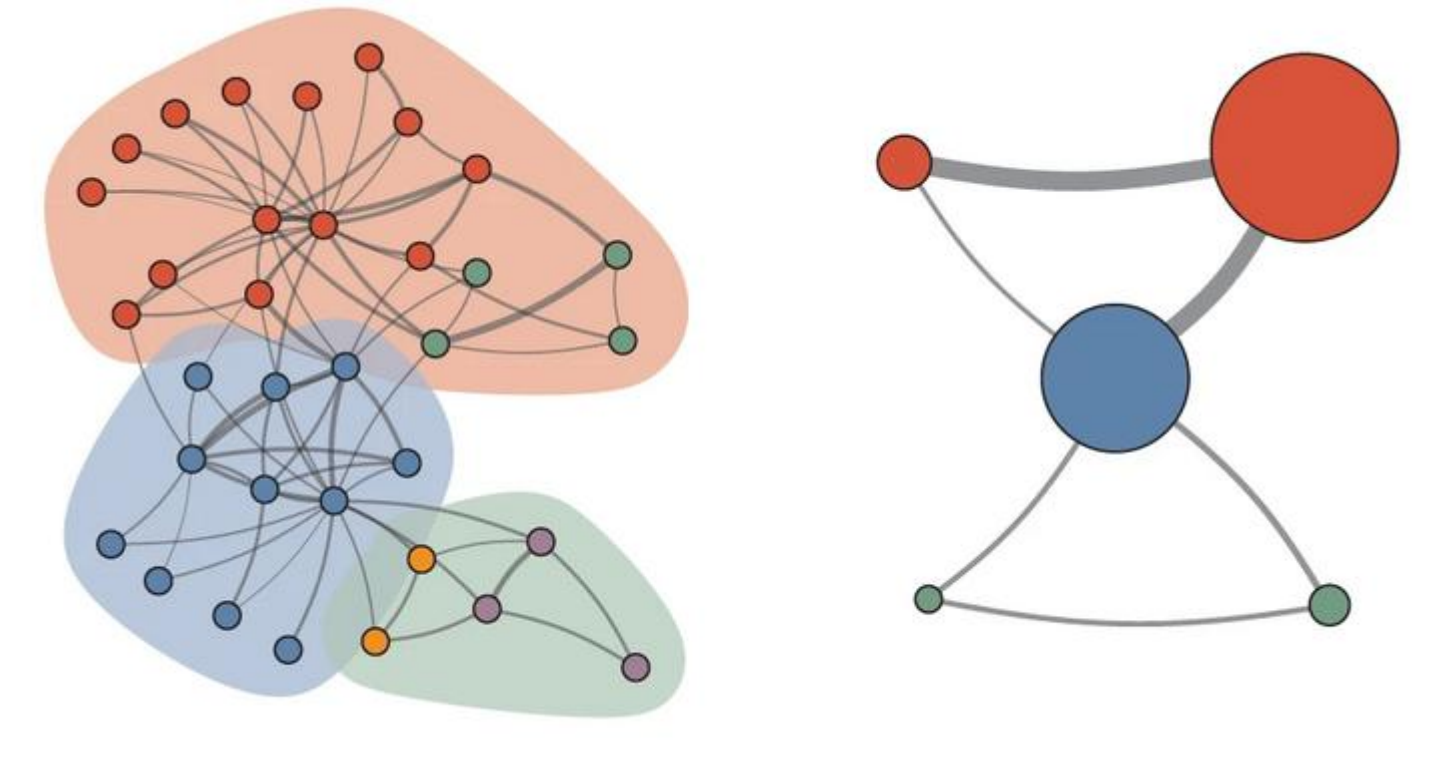
source

Advanced Visualization Tools: Leiden

- connecting each datapoint to its k nearest neighbors

$$d(n_i) = \sum_j A_{ij}$$

- calculate graph **partitions** from these connections
  → **communities** (= set of nodes, like a **circle of friends**)



source

Advanced Visualization Tools:

Leiden

Leiden is not just visualization – it is an actual clustering method:

1) load and scale the data as usual

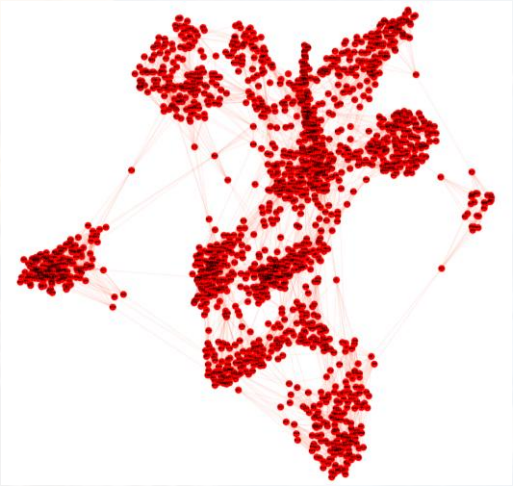
2) construct a graph with *k* nearest neighbors **(the kth nearest neighbors share an edge)**

3) run Leiden on the graph → returns communities, i.e. cluster = labels

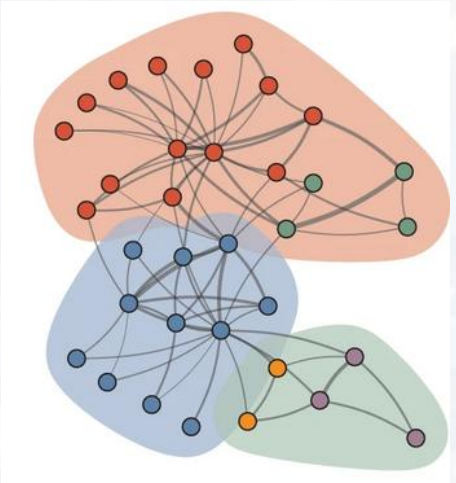
4) plot the clustered data using eg. UMAP

1)

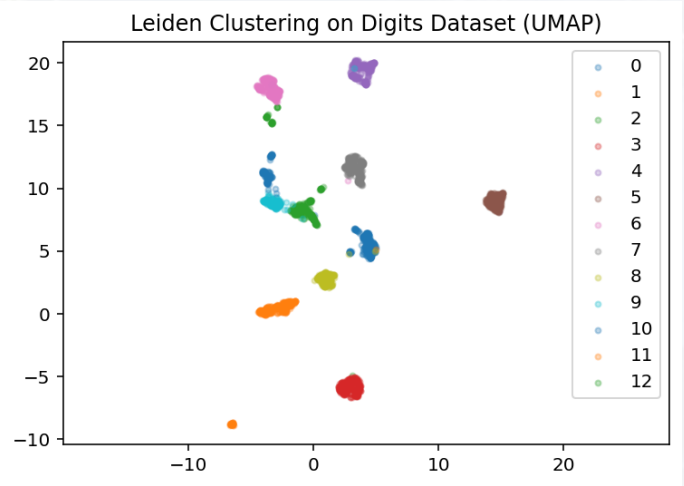
2)



3)



4)



Advanced Visualization Tools:

Leiden

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets      import load_digits
from sklearn.preprocessing import MinMaxScaler

digits   = load_digits()
XDigits  = digits.data
YDigits  = digits.target

MinMax   = MinMaxScaler()
XSDigits = MinMax.fit_transform(XDigits)
```

1) **load and scale the data as usual**
2) construct a graph with *k* nearest neighbors
3) run Leiden on the graph
4) plot the clustered data using eg. UMAP

Advanced Visualization Tools:

Leiden needs a graph in the first place.

We construct a graph by taking each data point (= **node**) and build **edges** with its **kth nearest** neighbors.

1) load and scale the data as usual
2) **construct a graph with *k* nearest neighbors**
3) run Leiden on the graph
4) plot the clustered data using eg. UMAP

```
import networkx as nx # pip install networkx
```

for visualizing a graph, not needed for analysis

```
from sklearn.neighbors import NearestNeighbors
```

for finding the nearest neighbors

```
k = 10  # number of neighbors

nbrs                = NearestNeighbors(n_neighbors = k).fit(XSDigits)
distances, indices = nbrs.kneighbors(XSDigits)

edges = [(i, j) for i, neigh in enumerate(indices) for j in neigh[1:]]
```

essentially, edges need to be a `list` of `tuples` containing the indices of those nodes that are connected

Advanced Visualization Tools:

Leiden needs a graph in the first place.

We construct a graph by taking each data point (= **node**) and build **edges** with its **kth nearest** neighbors.

1) load and scale the data as usual
2) **construct a graph with *k* nearest neighbors**
3) run Leiden on the graph
4) plot the clustered data using eg. UMAP

```
k = 10  # number of neighbors
nbrs                = NearestNeighbors(n_neighbors = k).fit(XSDigits)
distances, indices = nbrs.kneighbors(XSDigits)
edges = [(i, j) for i, neigh in enumerate(indices) for j in neigh[1:]]
```

```
My_G = nx.Graph()
My_G.add_nodes_from(YDigits)
My_G.add_edges_from(edges)
```

constructing a graph object for vis-ualization via nx

```
type(My_G)
networkx.classes.graph.Graph
```

Advanced Visualization Tools:

Leiden needs a graph in the first place.

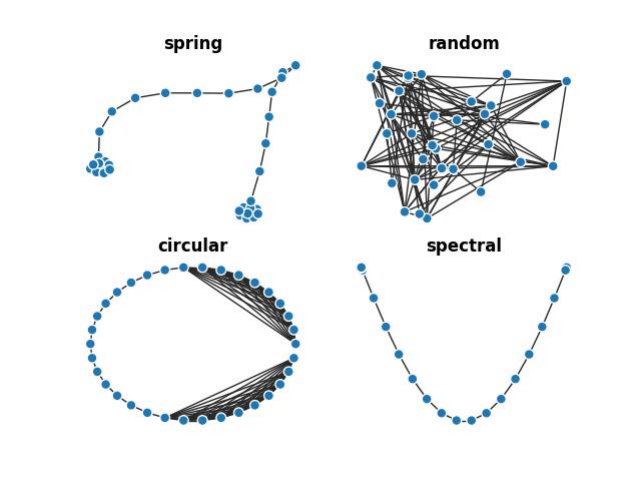
We construct a graph by taking each data point (= **node**) and build **edges** with its **kth nearest** neighbors.

1) load and scale the data as usual
2) **construct a graph with *k* nearest neighbors**
3) run Leiden on the graph
4) plot the clustered data using eg. UMAP

```
My_G = nx.Graph()
My_G.add_nodes_from(YDigits)
My_G.add_edges_from(edges)
```

visualizing the graph via nx

picking a graph layout




```
pos = nx.spring_layout(My_G)

plt.figure(figsize = (12, 12))

nx.draw_networkx_nodes(My_G, pos, node_color = 'red', node_size = 50)
nx.draw_networkx_labels(My_G, pos, font_size = 3, font_weight = 'bold')
nx.draw_networkx_edges(My_G, pos, alpha = 0.1, edge_color = '#ff5733')

plt.title("Graph")
plt.show()
```

Advanced Visualization Tools:

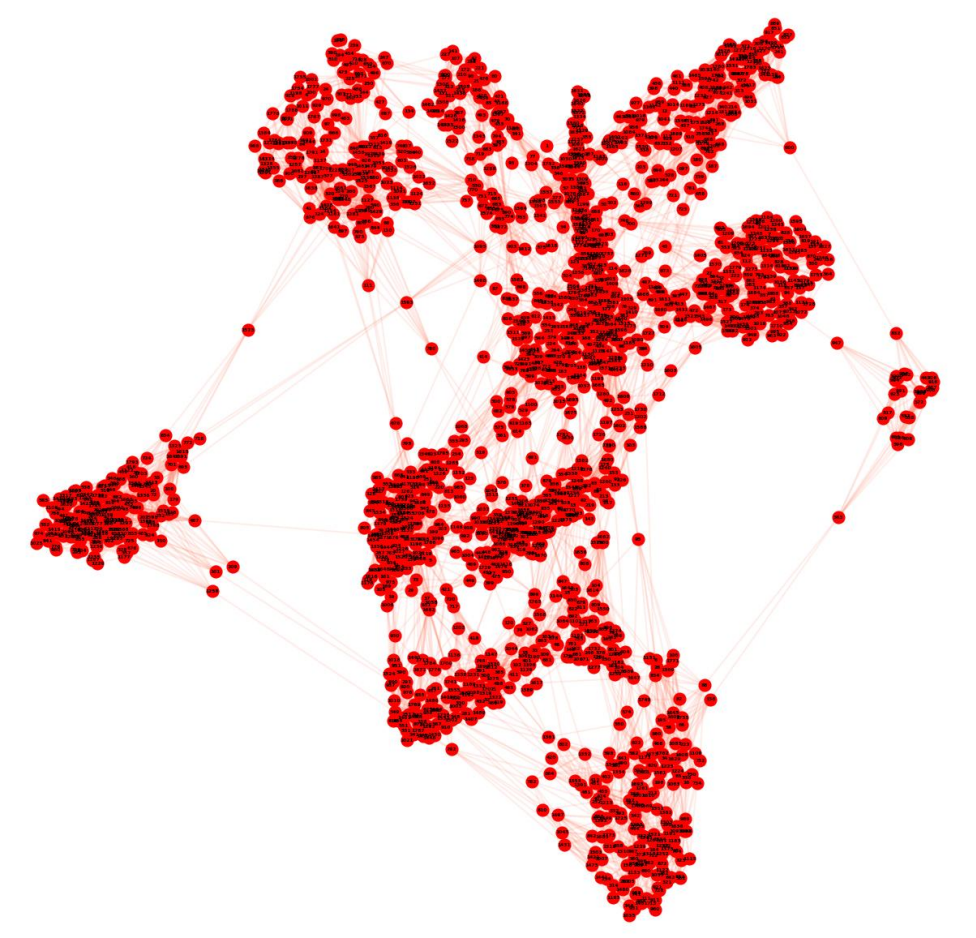
Leiden needs a graph in the first place.

We construct a graph by taking each data point (= **node**) and build **edges** with its **kth nearest** neighbors.

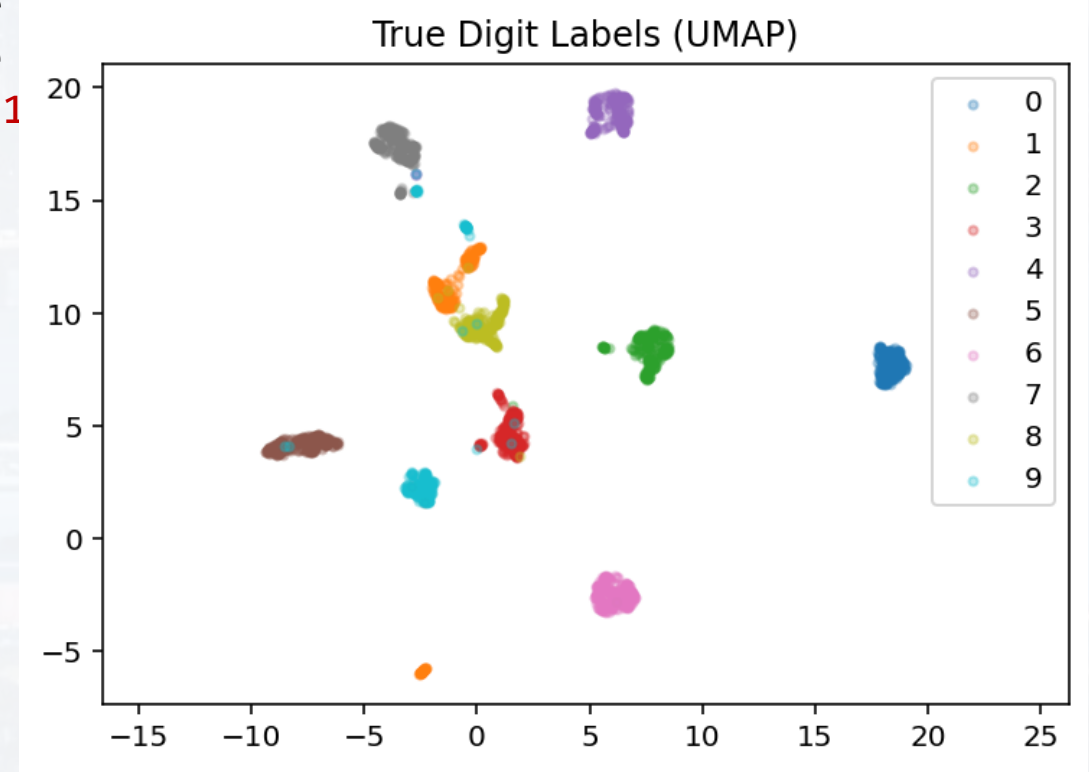
1) load and scale the data as usual
2) **construct a graph with *k* nearest neighbors**
3) run Leiden on the graph
4) plot the clustered data using eg. UMAP

Shows already surprisingly good clustering!

```
pos = nx.spring_layout(My_G)
```



```
lor
ize
 0.1
```



Advanced Visualization Tools: Leiden

```
import leidenalg     #pip install leidenalg
import igraph as ig #pip install igraph
```

1) load and scale the data as usual
2) construct a graph with *k* nearest neighbors
3) **run Leiden on the graph**
4) plot the clustered data using eg. UMAP

importing the `leidenalg` library for actual analysis and `igraph` for creating a graph object, that `leidenalg` understands

```
G = ig.Graph(edges = edges, directed = False)
G.simplify()
```

generating the graph object

performing clustering. **important**: the `resolution_parameter` defines how detailed the graph gets partitioned and therefore number of cluster can vary dramatically!

```
partition = leidenalg.find_partition(G,
                                     leidenalg.RBConfigurationVertexPartition,
                                     resolution_parameter = 1.0)
```

```
labels    = np.array(partition.membership)
print(f"Number of Leiden clusters: {len(np.unique(labels))}")
```

Advanced Visualization Tools: Leiden

1) load and scale the data as usual
2) construct a graph with *k* nearest neighbors
3) **run Leiden on the graph**
4) plot the clustered data using eg. UMAP

```
partition = leidenalg.find_partition(G,
                                     leidenalg.RBConfigurationVertexPartition,
                                     resolution_parameter = 1.0)

labels    = np.array(partition.membership)
print(f"Number of Leiden clusters: {len(np.unique(labels))}")
```

```
In [11]: partition = leidenalg.find_partition(G, leidenalg.RBConfigurationVertexPartition, resolution_parameter = 1.0)
    ...: labels     = np.array(partition.membership)
    ...: print(f"Number of Leiden clusters: {len(np.unique(labels))}")
Number of Leiden clusters: 12
```

```
In [13]: partition = leidenalg.find_partition(G, leidenalg.RBConfigurationVertexPartition, resolution_parameter = 10.0)
    ...: labels     = np.array(partition.membership)
    ...: print(f"Number of Leiden clusters: {len(np.unique(labels))}")
Number of Leiden clusters: 64
```

Advanced Visualization Tools:

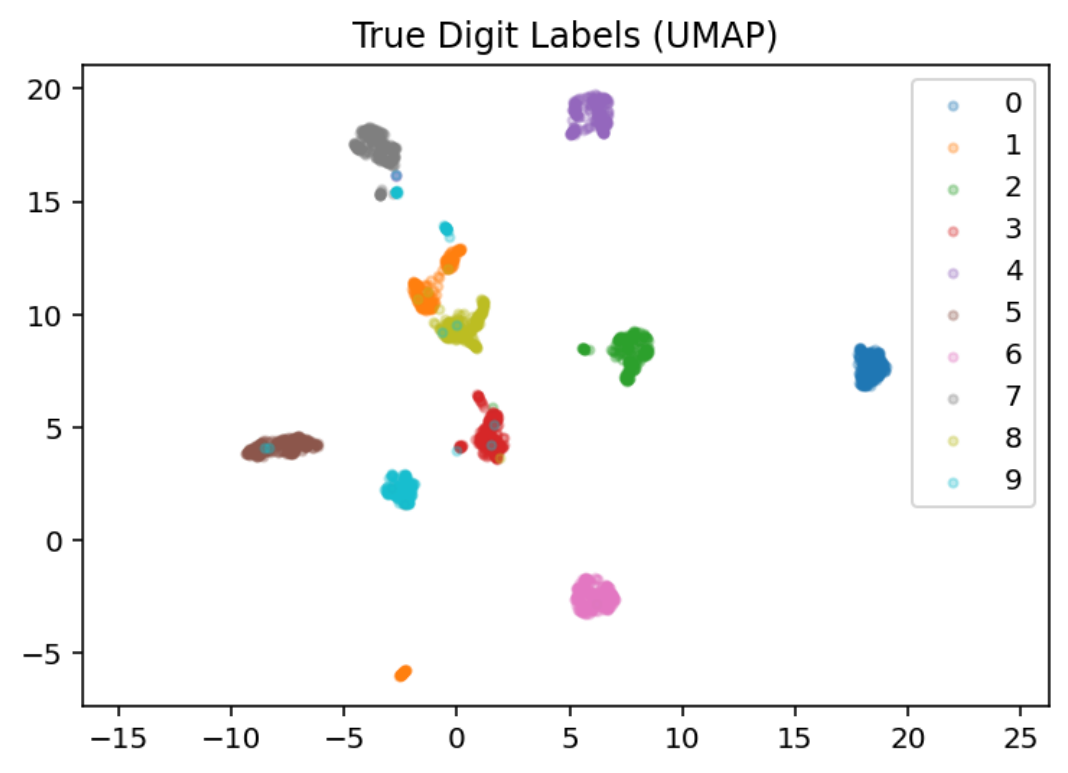
1) load and scale the data as usual
2) construct a graph with *k* nearest neighbors
3) run Leiden on the graph
4) **plot the clustered data using eg. UMAP**

```
labels        = np.array(partition.membership)


newXYDigits = umap.UMAP().fit_transform(XSDigits)


for i in range(np.max(labels) + 1):

    idx = np.argwhere(labels == i)
    plt.scatter(newXYDigits[idx, 0], newXYDigits[idx, 1], marker = '.',
                alpha = 0.3, label = str(i))

plt.gca().set_aspect('equal', 'datalim')
plt.title("Leiden Clustering on Digits Dataset (UMAP)", fontsize = 12)
plt.legend()
plt.show()
```

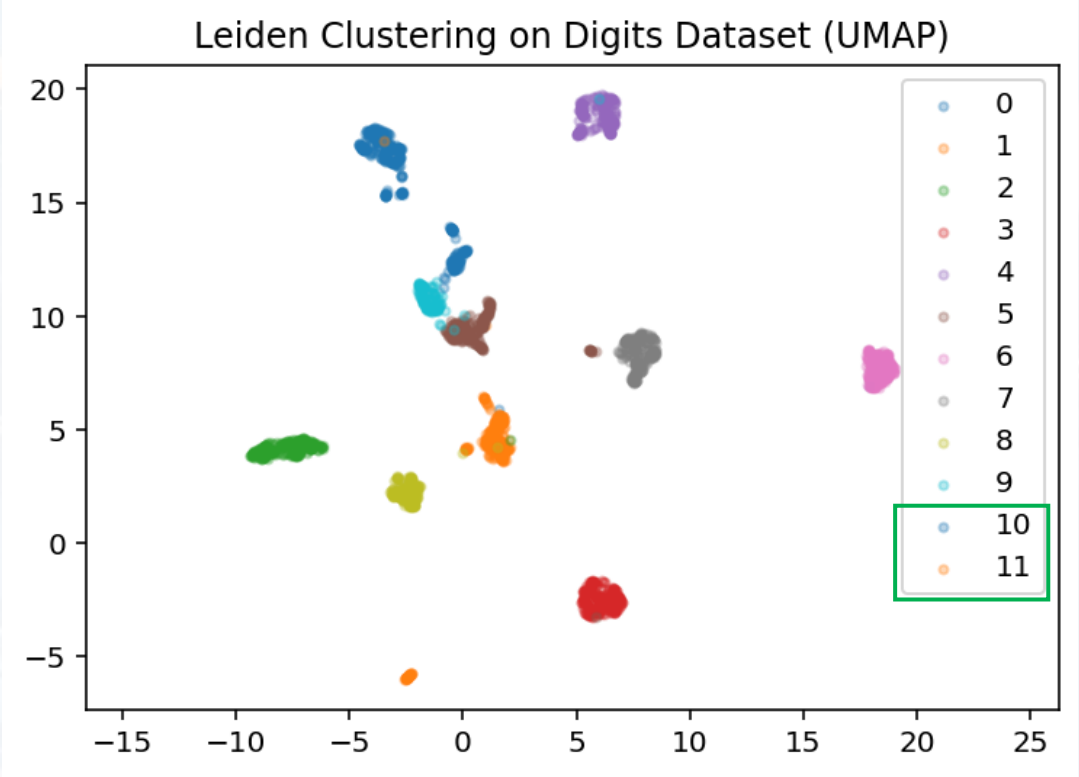
Advanced Visualization Tools: Leiden

1) load and scale the data as usual
2) construct a graph with *k* nearest neighbors
3) run Leiden on the graph
4) **plot the clustered data using eg. UMAP**

true clusters



Leiden clusters



note: Leiden labeling is arbitrary since it does not know the actual labels, but cluster should be monochromatic!

Advanced Visualization Tools

credit: David C. Lovelace

Outline

UMAP

t-SNE

Leiden

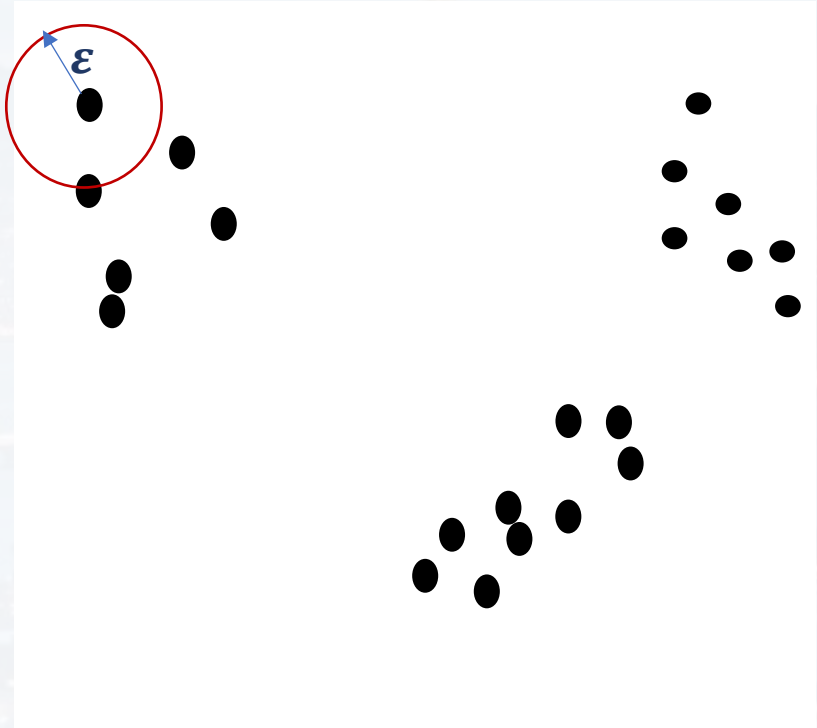
DBSCAN

Advanced Visualization Tools: DBSCAN

like Leiden: an actual unsupervised method

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

as before: - we are searching for k **neighboring points** within a certain **distance** ε
- constructing a graph based on these points



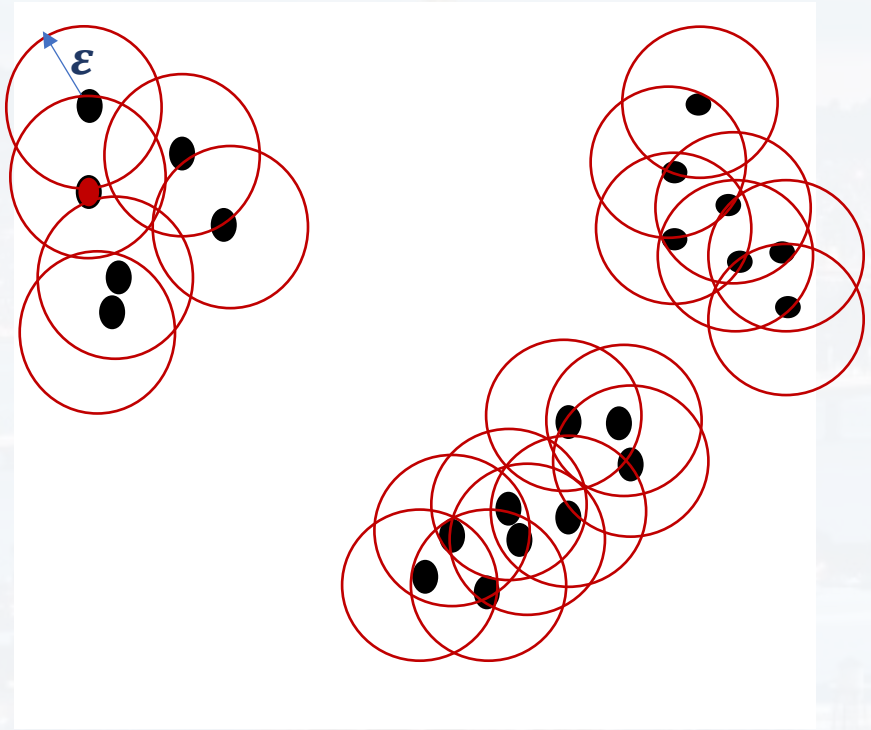
1) find all the points within d = ε

Advanced Visualization Tools: DBSCAN

like Leiden: an actual unsupervised method

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

as before: - we are searching for k **neighboring points** within a certain **distance** ε
- constructing a graph based on these points



1) find all the points within d = ε
   we do that for all the points

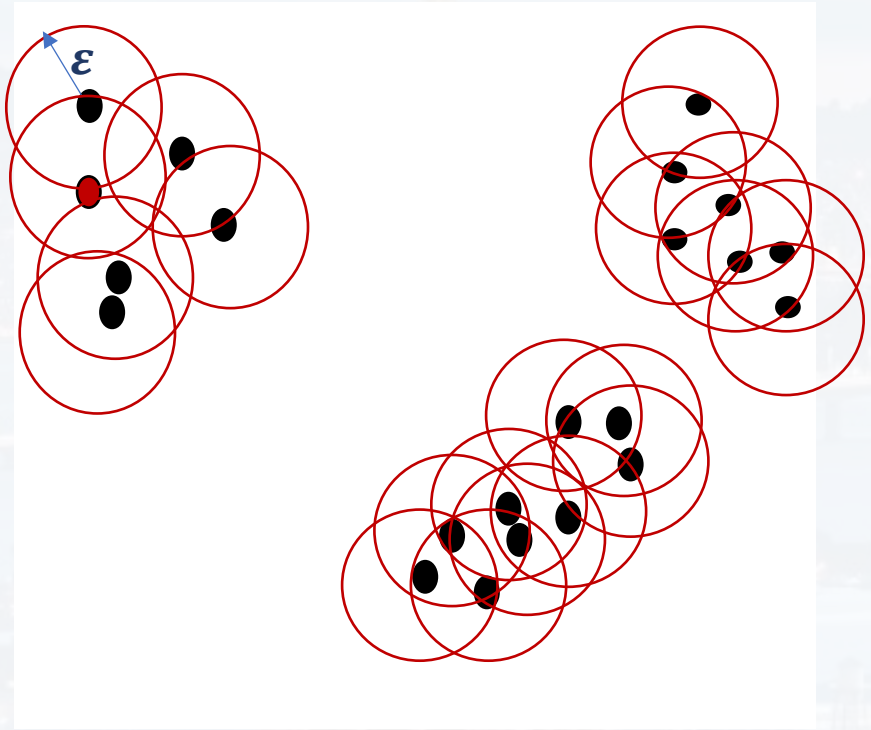
2) points with more than k neighbors are **core points**

Advanced Visualization Tools: DBSCAN

like Leiden: an actual unsupervised method

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

as before: - we are searching for k **neighboring points** within a certain **distance** ε
- constructing a graph based on these points



1) find all the points within d = ε
   we do that for all the points

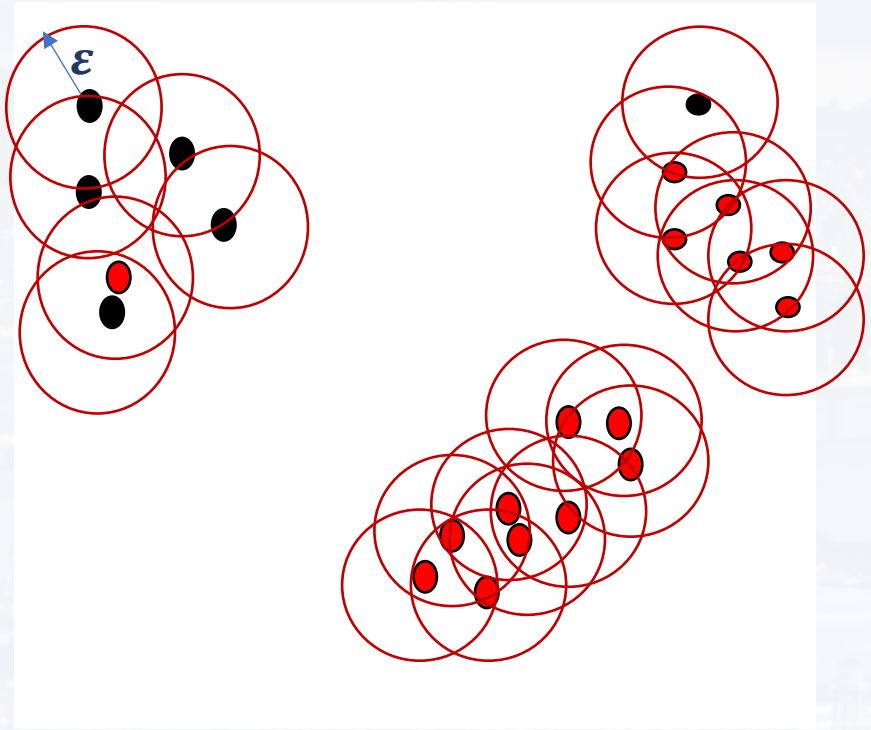
2) points with more than k neighbors are **core points**
   say $k = 2$

Advanced Visualization Tools: DBSCAN

like Leiden: an actual unsupervised method

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

as before: - we are searching for k **neighboring points** within a certain **distance** ε
- constructing a graph based on these points



1) find all the points within d = ε
   we do that for all the points

2) points with more than k neighbors are **core points**
   say $k = 2$

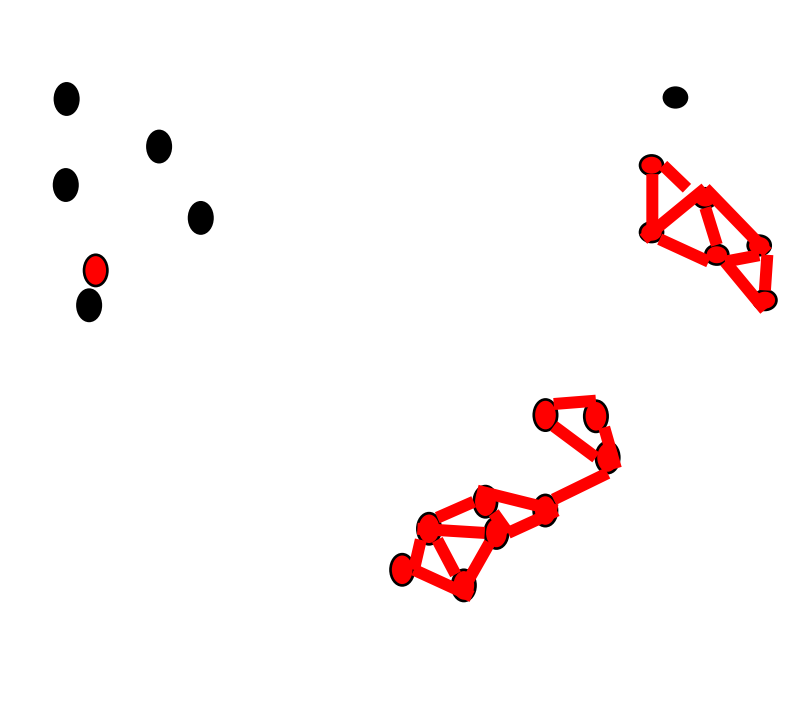
3) connect **core points** within d = ε

Advanced Visualization Tools: DBSCAN

like Leiden: an actual unsupervised method

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

as before: - we are searching for k **neighboring points** within a certain **distance** ε
- constructing a graph based on these points



1) find all the points within d = ε
   we do that for all the points

2) points with more than k neighbors are **core points**
   say $k = 2$

3) connect **core points** within d = ε

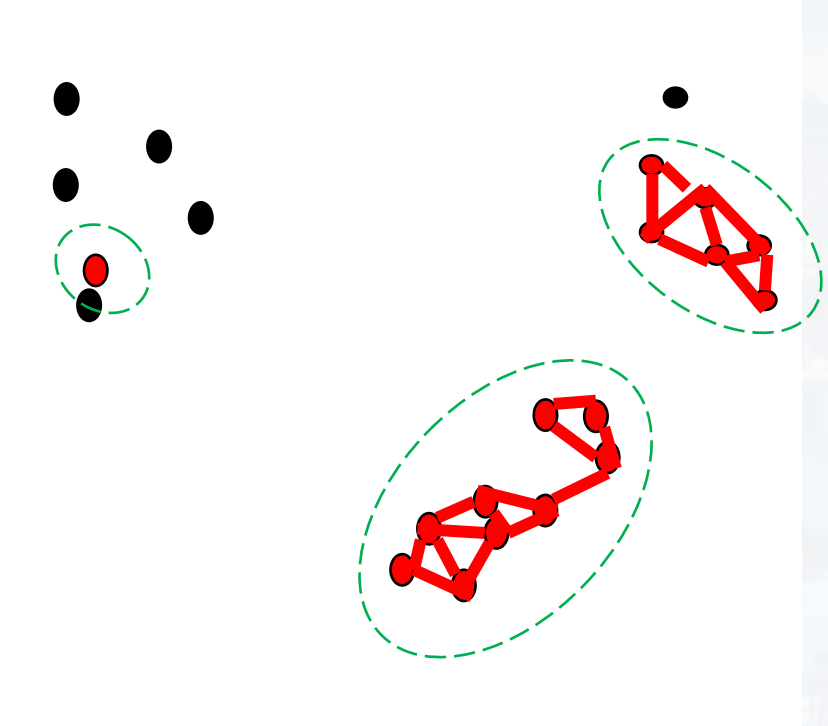
4) connected **core points** define a cluster

Advanced Visualization Tools: DBSCAN

like Leiden: an actual unsupervised method

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

as before: - we are searching for k **neighboring points** within a certain **distance** ε
- constructing a graph based on these points



1) find all the points within d = ε
   we do that for all the points

2) points with more than k neighbors are **core points**
   say $k = 2$

3) connect **core points** within d = ε

4) connected **core points** define a cluster

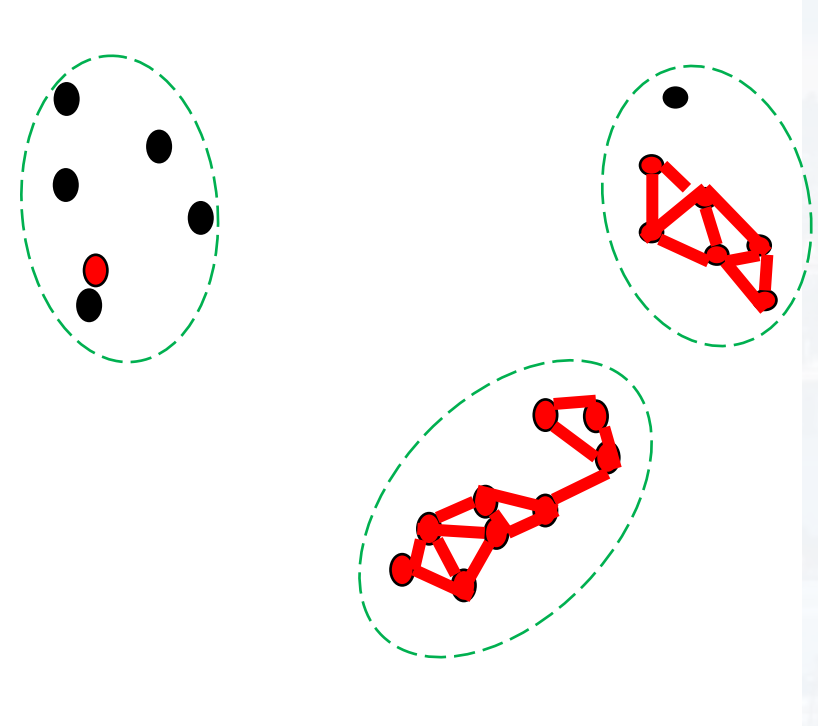
5) assign non-core points to their closest cluster

Advanced Visualization Tools: DBSCAN

like Leiden: an actual unsupervised method

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

as before: - we are searching for k **neighboring points** within a certain **distance** ε
- constructing a graph based on these points



1) find all the points within d = ε
   we do that for all the points

2) points with more than k neighbors are **core points**
   say $k = 2$

3) connect **core points** within d = ε

4) connected **core points** define a cluster

5) assign non-core points to their closest cluster

Advanced Visualization Tools: DBSCAN

We want to run DBSCAN on the digits dataset as before and visualize the clustering via UMAP:

```
from sklearn.cluster import DBSCAN

labels      = DBSCAN(eps = 1, min_samples = 7).fit(XSDigits).labels_


newXYDigits = umap.UMAP().fit_transform(XSDigits)


for i in range(np.max(labels) + 1):

    idx = np.argwhere(labels == i)
    plt.scatter(newXYDigits[idx, 0], newXYDigits[idx, 1], marker = '.',
                alpha = 0.3, label = str(i))

plt.gca().set_aspect('equal', 'datalim')
plt.title("DBSCAN Clustering on Digits Dataset (UMAP)", fontsize = 12)
plt.legend()
plt.show()
```

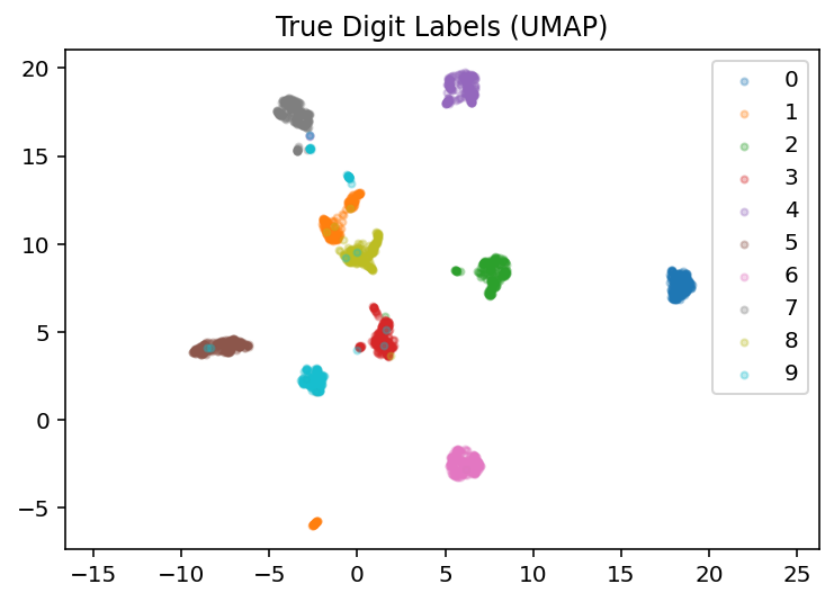
importing the DBSCAN

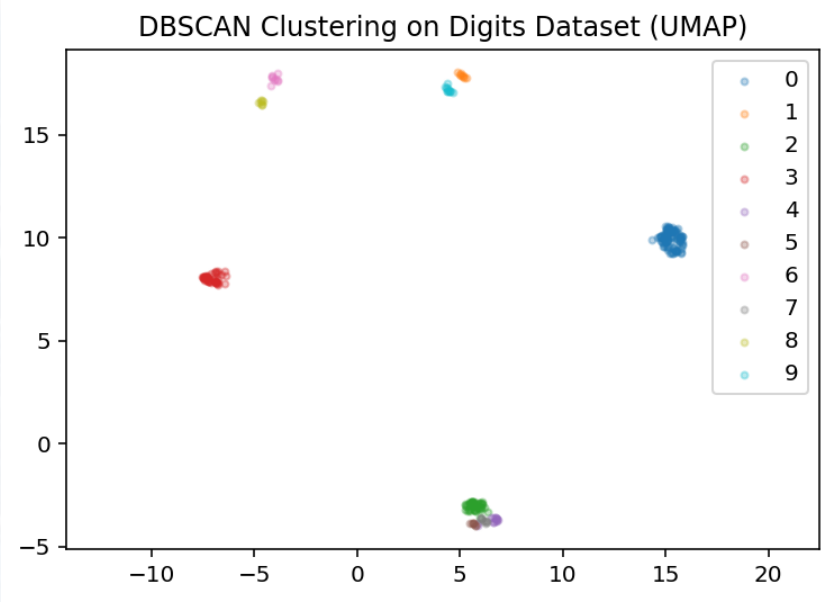
setting distance eps and minimum number of neighbors min_samples

Advanced Visualization Tools: DBSCAN

We want to run DBSCAN on the digits dataset as before and visualize the clustering via UMAP:

```
labels      = DBSCAN(eps = 1, min_samples = 7).fit(XSDigits).labels_
```





note I: DBSCAN is very sensitive to choices of distance `eps` and minimum number of neighbors `min_samples`
When data is normalized: choose $\varepsilon \leq 1.0$, default is $\varepsilon = 0.5$

note II: DBSCAN labeling is arbitrary since it does not know the actual labels, but cluster should be monochromatic!

Advanced Visualization Tools: Summary

method	what it does	important input arguments	pros	cons	notes
UMAP	maps high-dimensional data to 2D or 3D by approximating data manifold → visualization	`n_neighbors`: number of neighbors for approx manifold `min_dist`: distance for calculating probability that neighbors are actually connected	captures structure of highly non-linear manifolds	computationally intensive, cluster can have different appearance depending on `n_neighbors` and `min_dist`	alternative to PCA, but directions are hard to interpret
t-SNE	maps high-dimensional data to 2D or 3D by maintaining spatial distribution of data points → visualization	`perplexity`: influences probability that data points are considered neighbors	does not assume that data points are uniformly distributed on manifold	plots differ dramatically for different perplexity values	calculates probability that neighbors are actually connected via Gaussian kernel (therefore t-distributed) alternative to PCA, but directions are hard to interpret
Leiden	assigns data points to graph partitions according to common neighbors	`resolution_parameter`: how detailed the graph is being partitioned	data clouds are interpreted as graph (topological approach)	there is not natural value for the resolution parameter → number of cluster is variable	not a visualization method, a cluster method!
DBSCAN	assigns data points to cluster in a graph according to number of neighbors within a certain distance	`eps`: distance to find neighbors `min_samples`: min neighbors within `eps` in order to be regarded core point	simple, data clouds are interpreted as graph	result depends dramatically on `eps` and `min_samples`	not a visualization method, a cluster method!

Correlation, Dimension Reduction and PCA

Thank you very much for your attention!

credit: David C. Lovelace